NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY
POLITEHNICA BUCHAREST



BIOSINF Master Program

Artificial Intelligence III
Advanced Deep Learning Methods

Attack Mechanisms

Written course notes

Lecture by Ana Nicolae and Vlad Vasilescu

October 2026

These written notes accompany the course *Artificial Intelligence III: Advanced Deep Learning Methods.*

Contents

1 From prediction accuracy to adversarial robustness 1

1.1 Inputs, scores, and predictions . . . 1
1.2 Targeted and untargeted objectives . . . 1
1.3 What the attacker can observe and modify . . . 2

2 Optimization formulations and their geometry 3

2.1 Three ways to pose the attack problem . . . 3
2.2 What the norm measures . . . 4
2.3 The target region of a linear classifier . . . 5
2.3.1 An ℓ_1 attack as a linear program . . . 5
2.4 Minimum distance as a projection . . . 6

3 Constructing adversarial perturbations 7

3.1 The binary linear case . . . 7
3.1.1 Minimum ℓ_2 perturbation . . . 7
3.2 DeepFool: extending the Euclidean projection to nonlinear boundaries . . . 8
3.2.1 Extension to multiple classes . . . 9
3.3 From minimum norm to maximum allowable perturbation . . . 9
3.3.1 Minimum ℓ_∞ perturbation . . . 11
3.3.2 Maximum allowable perturbation with an ℓ_∞ budget . . . 11
3.4 Fast Gradient Sign Method . . . 12
3.4.1 The ℓ_2 version . . . 13
3.5 Iterative FGSM and projected gradient attacks . . . 13
3.6 Regularization-based attack . . . 15
3.7 Carlini–Wagner attack . . . 16
3.7.1 From the target constraint to a rectifier penalty . . . 16
3.7.2 Why the objective is convex for affine scores . . . 17
3.7.3 Computing the gradient and updating the input . . . 17

4 Can random noise produce an adversarial error? 18

4.1 A random direction and a favorable half-space . . . 18
4.2 A numerical example: averaging Gaussian components . . . 19
4.3 The general case and an explicit Gaussian tail bound . . . 19
4.4 From a tail probability to a classification error . . . 20
4.5 Geometric interpretation: the angle with the normal . . . 20

5 Additional attack mechanisms **21**

5.1 One Pixel Attack: sparsity as the constraint 21

5.2 ALMA: adapting the classification penalty 22

5.3 DDN: optimizing direction and radius separately 23

6 Attacks against generative models **24**

6.1 A common formulation and the attacker-controlled variables 24

6.2 Input and latent attacks on image generators 24

6.3 Diffusion models: conditioning and sampling randomness 25

6.4 Language models: jailbreaks and prompt injection 25

6.5 Training-time poisoning and backdoors 26

6.6 Privacy attacks and evaluation criteria 26

7 Comparing attacks and interpreting the results **26**

7.1 A consistent classification protocol 27

7.2 Questions for self-study 28

References **29**

1 From prediction accuracy to adversarial robustness

High accuracy on a test set describes how a model behaves on examples drawn from a particular data distribution. It does not, by itself, explain what happens when an input is deliberately modified to exploit the model. Adversarial robustness asks a local question: around an input that is correctly processed, how much can the input change before the output becomes wrong?

A natural disturbance, such as sensor noise or an occlusion, can also cause an error. The distinguishing feature of an adversarial perturbation is that it is *chosen to induce a failure*, subject to specified restrictions. The two problems are related, but resistance to ordinary noise is not sufficient evidence of resistance to an optimized perturbation. Equally, an adversarial example need not be visually imperceptible: a small norm is one useful mathematical model, not a complete definition of what a human can perceive.

The study by Szegedy and colleagues [16] helped establish adversarial examples as an important question for deep learning.

Linear classifiers already exhibit the geometry that makes adversarial examples possible. A point close to a separating hyperplane can cross it after a small, suitably directed change. Deep networks make the geometry more complex, but overfitting and nonlinearity are not necessary conditions for this phenomenon. The linear explanation also motivates efficient gradient attacks [5]. Whether an adversarial example exists depends on the input, model, allowed perturbations, and required output; it is not an unconditional property of every classifier at every input.

1.1 Inputs, scores, and predictions

Let $\mathbf{x}_0 \in \mathbb{R}^d$ be an original input with true class C_i. A classifier assigns scores $g_1(\mathbf{x}), \dots, g_k(\mathbf{x})$ to k possible classes and predicts

$$\widehat{\imath}(\mathbf{x}) = \operatorname*{arg\,max}_{j \in \{1,\dots,k\}} g_j(\mathbf{x}). \tag{1.1}$$

We initially assume $\widehat{\imath}(\mathbf{x}_0) = i$ so that a subsequent error can be attributed to the perturbation. Scores may be discriminant functions or pre-softmax logits. Their ordering determines the decision.

An attack is a mapping $\mathcal{A}$ that constructs a new input $\mathbf{x} = \mathcal{A}(\mathbf{x}_0)$. For the additive model,

$$\mathbf{x} = \mathbf{x}_0 + \mathbf{r}, \qquad \mathbf{r} \in \mathbb{R}^d. \tag{1.2}$$

For a fixed $\mathbf{r}$, this is a translation, hence an affine mapping. It is a linear operator if $\mathbf{r} = \mathbf{0}$. In practice the chosen perturbation usually depends on the original input and the classifier.

We denote the valid input domain by $\mathcal{X}$. For an image represented using intensities in $[0, 1]$, $\mathcal{X} = [0, 1]^d$. Remaining in $\mathbb{R}^d$ is not enough: a valid attack must also respect the allowed range and preserve the true label under the intended task. If the perturbation turns a genuine cat image into a genuine dog image, predicting "dog" is not an adversarial error.

1.2 Targeted and untargeted objectives

A *targeted* attack aims for a specified class C_t, with $t \neq i$. The target must score at least as highly as every alternative:

$$g_t(\mathbf{x}) \geq g_j(\mathbf{x}) \quad \text{for all } j \neq t. \tag{1.3}$$

Define the targeted margin

$$\phi_t(\mathbf{x}) = \max_{j \neq t} g_j(\mathbf{x}) - g_t(\mathbf{x}), \qquad \Omega_t = \{\mathbf{x} \in \mathcal{X} : \phi_t(\mathbf{x}) \leq 0\}. \tag{1.4}$$

Symbol	Meaning
$\mathbf{x}_0, \mathbf{x}, \mathbf{r}$	Original input, perturbed input, and additive perturbation.
d, k	Input dimension and number of classes.
C_i, C_t	True/original class and chosen target class, $t \neq i$.
$g_j(\mathbf{x})$	Score associated with class C_j.
$\phi_t(\mathbf{x}), \Omega_t$	Targeted score margin and target feasible region.
η	Total allowed perturbation magnitude in a specified norm.
λ	Weight of a penalty term in a regularized objective.
α	Step length; its local meaning is stated where it is used.
$J(\mathbf{x}; \mathbf{w})$	Attack loss, differentiated with respect to the input.
P_Ω	Euclidean projection onto a set Ω.
M	Number of permitted model queries in a black-box attack.

Table 1: Core notation retained from the lecture. Iterations use ℓ in place of the slides' superscript k to distinguish them from the class count.

The margin is positive before the target overtakes its strongest competitor, zero at a tie, and negative when the target is the unique winner.

An *untargeted* attack only needs some class other than C_i to win. Thus

$$\Omega_{\mathrm{u}} = \left\{ \mathbf{x} \in \mathcal{X} : g_i(\mathbf{x}) - \max_{j \neq i} g_j(\mathbf{x}) \leq 0 \right\}. \tag{1.5}$$

The maximum is essential: at least one competing score must exceed the original score. Replacing it by a minimum would require *every* competitor to exceed the original class, which is a much stronger and different requirement.

These non-strict inequalities include decision ties. For mathematical distance calculations this closed-set convention is convenient. To ensure an actual targeted prediction, one may require $\phi_t(\mathbf{x}) \leq -\kappa$ for a margin $\kappa > 0$, or explicitly check the model's tie-breaking rule. The same care applies to untargeted success.

1.3 What the attacker can observe and modify

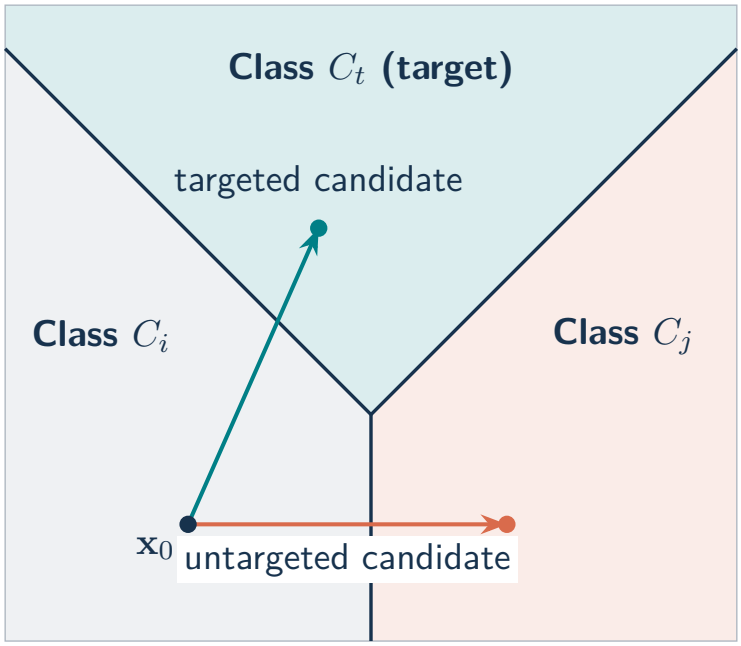
The *threat model* specifies the desired failure, controllable variables, available information, and resource limits. In a white-box setting the attacker can evaluate the model and differentiate its scores or loss. In a black-box setting the internal model is unavailable, but a query interface may return scores, probabilities, or only a predicted label. These provide different amounts of information. A gray-box setting gives partial knowledge, such as the architecture or some training information.

With at most M queries, the observations may be used to fit surrogate scores $\widetilde{g}_j$. For example, the surrogate targeted region is

$$\widetilde{\Omega}_t = \left\{ \mathbf{x} \in \mathcal{X} : \max_{j \neq t} \widetilde{g}_j(\mathbf{x}) - \widetilde{g}_t(\mathbf{x}) \leq 0 \right\}. \tag{1.6}$$

This is an estimate of the victim's region, not a replacement for it. A candidate found using the surrogate must be tested on the victim model. A label-only interface does not directly reveal the discriminant functions.

The core of this course concerns *evasion*: modifying an input while a trained model is fixed. Poisoning changes training data, and a backdoor seeks trigger-dependent behavior in a compromised model. These different intervention points will reappear in the discussion of generative networks.



$$g_i(\mathbf{x}) = -x_1, \quad g_j(\mathbf{x}) = x_1, \quad g_t(\mathbf{x}) = x_2$$
$$\widehat{i}(\mathbf{x}) = \arg\max_{q\in\{i,j,t\}} g_q(\mathbf{x})$$

Figure 1: A targeted perturbation must reach the chosen region. An untargeted perturbation can reach any competing region. The admissible directions therefore depend on the attack objective.

2 Optimization formulations and their geometry

2.1 Three ways to pose the attack problem

With the target set specified, the next question is how to select a perturbation. The course uses three complementary formulations. They are written below in the original slide notation, where $\|\cdot\|$ denotes any chosen norm. In applications, the input-validity constraint $\mathbf{x} \in \mathcal{X}$ is added when needed.

Definition 2.1 (Minimum-norm attack)**.** Seek the closest point in the target region:

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \|\mathbf{x} - \mathbf{x}_0\| \\ \text{subject to} \quad & \max_{j\neq t}\{g_j(\mathbf{x})\} - g_t(\mathbf{x}) \leq 0, \end{aligned} \tag{2.1}$$

where $\|\cdot\|$ is the prescribed norm.

The distance of a successful solution quantifies how much change was sufficient to reach C_t. The target may be unreachable within the valid input domain; then the feasible set is empty. An untargeted version replaces ϕ_t by $g_i - \max_{j\neq i} g_j$ in the constraint.

Definition 2.2 (Maximum-allowable attack)**.** Fix an admissible radius $\eta > 0$ and use it to reduce the target margin:

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & \max_{j\neq t}\{g_j(\mathbf{x})\} - g_t(\mathbf{x}) \\ \text{subject to} \quad & \|\mathbf{x} - \mathbf{x}_0\| \leq \eta, \end{aligned} \tag{2.2}$$

where $\|\cdot\|$ is the prescribed norm and $\eta > 0$.

The term "maximum allowable" refers to the perturbation budget, not a requirement to use every part of that budget. A negative optimal margin implies targeted success. A positive *global* optimum rules it out within the stated region. A numerical optimizer returning a positive margin does not establish the global optimum for a neural network.

Definition 2.3 (Regularization-based attack)**.** Balance distortion and the target margin:

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{x}-\mathbf{x}_0\| + \lambda \left(\max_{j \neq t} \{g_j(\mathbf{x})\} - g_t(\mathbf{x}) \right), \tag{2.3}$$

where $\|\cdot\|$ is the prescribed norm and $\lambda > 0$ is the regularization parameter.

This replaces the attack-success constraint by a term in the objective. A large λ emphasizes reaching the target, whereas a small value emphasizes remaining close to the original input. The result need not be successful. Later we will replace the raw margin by a hinge penalty and, where appropriate, use a squared Euclidean distance.

These formulations explore the same tradeoff between distortion and attack effectiveness. Under suitable convexity and duality assumptions, corresponding choices of budget and multiplier can identify common solutions. They are not automatically equivalent for an arbitrary nonlinear network or arbitrary parameter values. This distinction matters when comparing attack algorithms.

2.2 What the norm measures

For $\mathbf{r} = (r_1, \ldots, r_d)^\top$, the most common choices are

$$\|\mathbf{r}\|_1 = \sum_{j=1}^{d} |r_j|, \qquad \|\mathbf{r}\|_2 = \left(\sum_{j=1}^{d} r_j^2 \right)^{1/2}, \qquad \|\mathbf{r}\|_\infty = \max_j |r_j|. \tag{2.4}$$

The quantity $\|\mathbf{r}\|_0 = \#\{j : r_j \neq 0\}$ counts changed coordinates. It is often called the "ℓ_0 norm," although it is not a norm: multiplying a nonzero vector by a nonzero scalar leaves this count unchanged.

An ℓ_1 constraint limits total absolute change and can favor sparse solutions. An ℓ_2 constraint limits total perturbation energy. An ℓ_∞ constraint limits the change in every coordinate separately. Thus an ℓ_∞ perturbation can touch many pixels while making only a small change to each one. In a color image, counting changed pixels is different from counting changed channel coordinates.

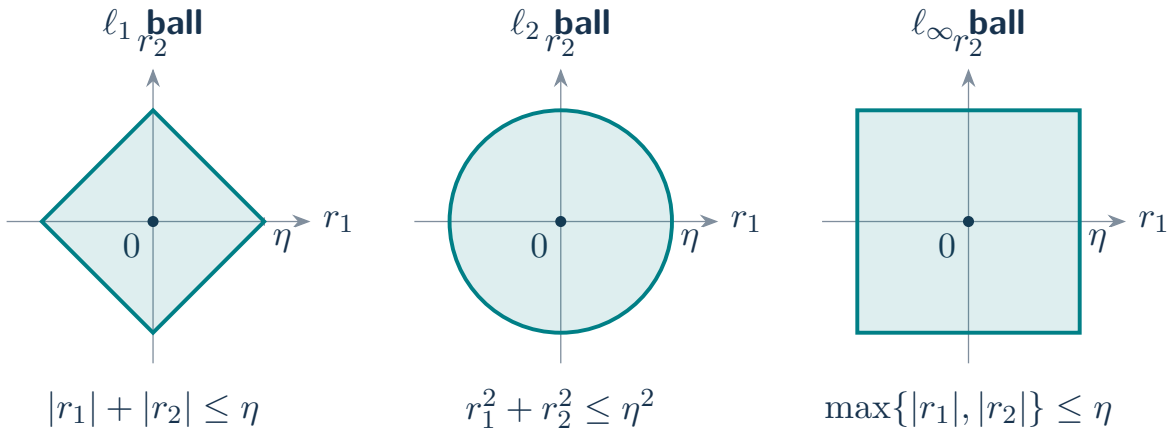


Figure 2: Two-dimensional unit balls for three norms. Translating these sets to $\mathbf{x}_0$ and enlarging them until they meet the target region gives a geometric interpretation of minimum-norm attacks.

The inequalities

$$\|\mathbf{r}\|_\infty \le \|\mathbf{r}\|_2 \le \|\mathbf{r}\|_1, \qquad \|\mathbf{r}\|_2 \le \sqrt{d}\,\|\mathbf{r}\|_\infty \tag{2.5}$$

show why a budget value cannot be compared across norms without context. Input scaling also matters: a radius on $[0, 255]$ intensities is numerically different from the corresponding radius on $[0, 1]$ intensities.

A weighted distance $\|\mathbf{M}(\mathbf{x} - \mathbf{x}_0)\|_2$ can emphasize selected features. However, setting a weight to zero merely makes changes in that direction unpenalized. To forbid changes outside a region, impose explicit constraints such as $r_j = 0$ outside the allowed support.

2.3 The target region of a linear classifier

Suppose the score functions are affine:

$$g_j(\mathbf{x}) = \mathbf{w}_j^\top \mathbf{x} + w_{j,0}, \qquad j = 1, \dots, k. \tag{2.6}$$

The requirement $g_j(\mathbf{x}) \le g_t(\mathbf{x})$ becomes

$$(\mathbf{w}_j - \mathbf{w}_t)^\top \mathbf{x} \le w_{t,0} - w_{j,0}. \tag{2.7}$$

Let $\mathbf{A} \in \mathbb{R}^{d\times(k-1)}$ have columns $\mathbf{a}_j = \mathbf{w}_j - \mathbf{w}_t$ for $j \ne t$, and let $\mathbf{b} \in \mathbb{R}^{k-1}$ have corresponding entries $b_j = w_{t,0} - w_{j,0}$. Then

$$\Omega_t = \{\mathbf{x} \in \mathcal{X} : \mathbf{A}^\top \mathbf{x} \le \mathbf{b}\}. \tag{2.8}$$

Without the input-domain restriction, this is an intersection of closed half-spaces: a closed convex polyhedron. It need not be bounded or even nonempty. When $\mathcal{X} = [0, 1]^d$, a nonempty intersection is bounded.

The boundaries are fixed once the classifier is fixed. What can change as a point moves is the *active competitor*, $j^*(\mathbf{x}) \in \arg\max_{j\ne t} g_j(\mathbf{x})$. Freezing this index at its value at $\mathbf{x}_0$ generally loses some of the multiclass constraints.

For a linear classifier with a polyhedral input domain such as $[0, 1]^d$ or $\mathbb{R}^d$, minimizing $\frac{1}{2}\|\mathbf{x} - \mathbf{x}_0\|_2^2$ over (2.8) is a convex quadratic program. Squaring the nonnegative norm does not change its minimizers. Other norms produce different convex programs; a norm objective is not automatically a quadratic objective.

2.3.1 An ℓ_1 attack as a linear program

Write $\mathbf{r} = \mathbf{x} - \mathbf{x}_0$ and $\widetilde{\mathbf{b}} = \mathbf{b} - \mathbf{A}^\top \mathbf{x}_0$. Introduce nonnegative vectors $\mathbf{r}^+, \mathbf{r}^-$ with $\mathbf{r} = \mathbf{r}^+ - \mathbf{r}^-$. The attack is equivalently expressed as

$$\begin{aligned} \min_{\mathbf{r}^+,\mathbf{r}^-} \quad & \mathbf{1}^\top(\mathbf{r}^+ + \mathbf{r}^-) \\ \text{subject to} \quad & [\mathbf{A}^\top \; -\mathbf{A}^\top] \begin{bmatrix} \mathbf{r}^+ \\ \mathbf{r}^- \end{bmatrix} \le \widetilde{\mathbf{b}}, \\ & \mathbf{r}^+ \ge \mathbf{0}, \quad \mathbf{r}^- \ge \mathbf{0}. \end{aligned} \tag{2.9}$$

If the image domain is enforced, add $-\mathbf{x}_0 \le \mathbf{r}^+ - \mathbf{r}^- \le \mathbf{1} - \mathbf{x}_0$. The scalar objective sums the positive and negative parts. At an optimum both parts cannot be positive in the same coordinate: reducing them by the same amount preserves $\mathbf{r}$ and lowers the objective. Consequently their sum yields the desired ℓ_1 norm.

Example 2.1 (A two-dimensional numerical problem)**.** Consider an input $\mathbf{x} = (x_1, x_2)^\top \in [0,1]^2$ and three affine scores

$$g_1(\mathbf{x}) = \tfrac{3}{8} - 2x_1 + x_2, \qquad g_2(\mathbf{x}) = -\tfrac{3}{4} - x_1 + 2x_2, \qquad g_3(\mathbf{x}) = 0. \tag{2.10}$$

At the clean input $\mathbf{x}_0 = (\frac{1}{4}, \frac{3}{4})^\top$, the scores are $(\frac{5}{8}, \frac{1}{2}, 0)$, so the predicted class is C_1. We seek the smallest ℓ_1 perturbation reaching the closed target region for C_3. The inequalities $g_1(\mathbf{x}) \le g_3(\mathbf{x})$ and $g_2(\mathbf{x}) \le g_3(\mathbf{x})$ give

$$\mathbf{A}^\top = \begin{bmatrix} -2 & 1 \\ -1 & 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} -\frac{3}{8} \\ \frac{3}{4} \end{bmatrix}, \qquad \widetilde{\mathbf{b}} = \mathbf{b} - \mathbf{A}^\top \mathbf{x}_0 = \begin{bmatrix} -\frac{5}{8} \\ -\frac{1}{2} \end{bmatrix}. \tag{2.11}$$

With $\mathbf{r} = \mathbf{r}^+ - \mathbf{r}^-$, the explicit linear program is

$$\begin{aligned} \min_{\mathbf{r}^+, \mathbf{r}^- \ge \mathbf{0}} \quad & r_1^+ + r_2^+ + r_1^- + r_2^- \\ \text{subject to} \quad & \begin{bmatrix} -2 & 1 & 2 & -1 \\ -1 & 2 & 1 & -2 \end{bmatrix} \begin{bmatrix} r_1^+ \\ r_2^+ \\ r_1^- \\ r_2^- \end{bmatrix} \le \begin{bmatrix} -\frac{5}{8} \\ -\frac{1}{2} \end{bmatrix}, \\ & \begin{bmatrix} -\frac{1}{4} \\ -\frac{3}{4} \end{bmatrix} \le \mathbf{r}^+ - \mathbf{r}^- \le \begin{bmatrix} \frac{3}{4} \\ \frac{1}{4} \end{bmatrix}. \end{aligned} \tag{2.12}$$

The final bounds keep the perturbed input inside $[0,1]^2$. Adding the two target constraints yields $-3r_1 + 3r_2 \le -\frac{9}{8}$. Thus every feasible perturbation satisfies

$$|r_1| + |r_2| \ge r_1 - r_2 \ge \tfrac{3}{8}. \tag{2.13}$$

This lower bound is attained by

$$\mathbf{r}^{+*} = \begin{bmatrix} \frac{1}{4} \\ 0 \end{bmatrix}, \quad \mathbf{r}^{-*} = \begin{bmatrix} 0 \\ \frac{1}{8} \end{bmatrix}, \quad \mathbf{r}^* = \begin{bmatrix} \frac{1}{4} \\ -\frac{1}{8} \end{bmatrix}, \quad \mathbf{x}^* = \begin{bmatrix} \frac{1}{2} \\ \frac{5}{8} \end{bmatrix}. \tag{2.14}$$

Consequently, the minimum distortion is $\|\mathbf{r}^*\|_1 = \frac{3}{8}$. Both target constraints are active. Any other optimum would also have to make both constraints equalities, whose unique intersection is $\mathbf{r}^*$; hence this solution is unique.

At $\mathbf{x}^*$ all three scores equal zero. Reaching the closed target region therefore does not by itself guarantee class C_3 under an arbitrary tie rule. A small overshoot, for example

$$\mathbf{x}' = \mathbf{x}_0 + \tfrac{11}{10}\mathbf{r}^* = \begin{bmatrix} \frac{21}{40} \\ \frac{49}{80} \end{bmatrix}, \tag{2.15}$$

remains inside $[0,1]^2$ and gives scores $(g_1(\mathbf{x}'), g_2(\mathbf{x}'), g_3(\mathbf{x}')) = (-\frac{1}{16}, -\frac{1}{20}, 0)$. Class C_3 now wins strictly, at the slightly larger distortion $\|\mathbf{x}' - \mathbf{x}_0\|_1 = \frac{33}{80}$.

2.4 Minimum distance as a projection

The Euclidean minimum-norm problem is a projection problem:

$$\mathbf{x}^* \in \arg\min_{\mathbf{x} \in \Omega_t} \|\mathbf{x} - \mathbf{x}_0\|_2. \tag{2.16}$$

For a nonempty closed convex set, this projection exists and is unique, and we write $\mathbf{x}^* = P_{\Omega_t}(\mathbf{x}_0)$. A closed nonconvex region may have several equally close points, in which case the projection

notation denotes a set of minimizers. If $\mathbf{x}_0$ is outside the target set, a nearest point lies on its boundary; if it is already inside, the minimum perturbation is zero.

At a smooth boundary point reached without an active input-domain constraint, the Euclidean displacement is normal to the boundary. At a corner of a convex feasible set, $\mathbf{x}_0 - \mathbf{x}^*$ belongs to the outward normal cone, and need not align with only one class-separating normal. For other norms the first-contact geometry changes, as the norm-ball figure suggests.

The lecture parameterization

$$\mathbf{x}(\alpha) = \mathbf{x}_0 + \alpha(\mathbf{x}^* - \mathbf{x}_0) \tag{2.17}$$

makes step length explicit after a nearest point $\mathbf{x}^*$ is selected. Here $\alpha = 0$ gives the clean input and $\alpha = 1$ reaches the projection. A small overshoot can resolve a decision tie, but a large one may leave the chosen class region or the valid input domain. This is especially relevant for a multiclass or nonlinear classifier.

3 Constructing adversarial perturbations

3.1 The binary linear case

Consider two classes C_i and C_t with affine scores. Set

$$\mathbf{w} = \mathbf{w}_i - \mathbf{w}_t, \qquad w_0 = w_{i,0} - w_{t,0}, \qquad g(\mathbf{x}) = g_i(\mathbf{x}) - g_t(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0. \tag{3.1}$$

Assume $\mathbf{w} \neq \mathbf{0}$ and define the positive clean margin

$$m = g(\mathbf{x}_0) = \mathbf{w}^\top \mathbf{x}_0 + w_0 > 0. \tag{3.2}$$

The original point lies on the C_i side; reaching the separating hyperplane requires $\mathbf{w}^\top \mathbf{r} = -m$. For now, we work in $\mathbb{R}^d$ without image-box constraints. The formulas are exact under this assumption. If their output leaves $\mathcal{X}$, the constrained problem must be solved instead; clipping an answer can destroy the desired crossing.

3.1.1 Minimum ℓ_2 perturbation

For two classes, the target condition $g_t(\mathbf{x}) \geq g_i(\mathbf{x})$ reduces to $\mathbf{w}^\top \mathbf{x} + w_0 \leq 0$. We replace this inequality by an equality because the closest point to $\mathbf{x}_0$ in the target half-space lies on the decision boundary. The adversarial attack can therefore be formulated as

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{x} - \mathbf{x}_0\|_2 \qquad \text{subject to} \quad \mathbf{w}^\top \mathbf{x} + w_0 = 0. \tag{3.3}$$

Proposition 3.1. *The solution of this optimization problem is*

$$\boxed{\mathbf{x}^* = \mathbf{x}_0 - \frac{\mathbf{w}^\top \mathbf{x}_0 + w_0}{\|\mathbf{w}\|_2} \frac{\mathbf{w}}{\|\mathbf{w}\|_2}.} \tag{3.4}$$

Proof. The Lagrangian of the constrained optimization is given by

$$L(\mathbf{x}, \lambda) = \tfrac{1}{2}\|\mathbf{x} - \mathbf{x}_0\|_2^2 + \lambda(\mathbf{w}^\top \mathbf{x} + w_0). \tag{3.5}$$

The solution of the optimization is the saddle point $(\mathbf{x}^*, \lambda^*)$ such that $\nabla_{\mathbf{x}} L = \mathbf{0}$ and $\nabla_\lambda L = 0$. Taking derivatives with respect to $\mathbf{x}$ and λ yields

$$\nabla_{\mathbf{x}} L = \mathbf{x} - \mathbf{x}_0 + \lambda \mathbf{w} = \mathbf{0}, \tag{3.6}$$

$$\nabla_\lambda L = \mathbf{w}^\top \mathbf{x} + w_0 = 0. \tag{3.7}$$

Multiplying the first equation by $\mathbf{w}^\top$ yields

$$0 = \mathbf{w}^\top \mathbf{x} - \mathbf{w}^\top \mathbf{x}_0 + \lambda \|\mathbf{w}\|_2^2 = -w_0 - \mathbf{w}^\top \mathbf{x}_0 + \lambda \|\mathbf{w}\|_2^2. \tag{3.8}$$

Thus, the optimal λ is

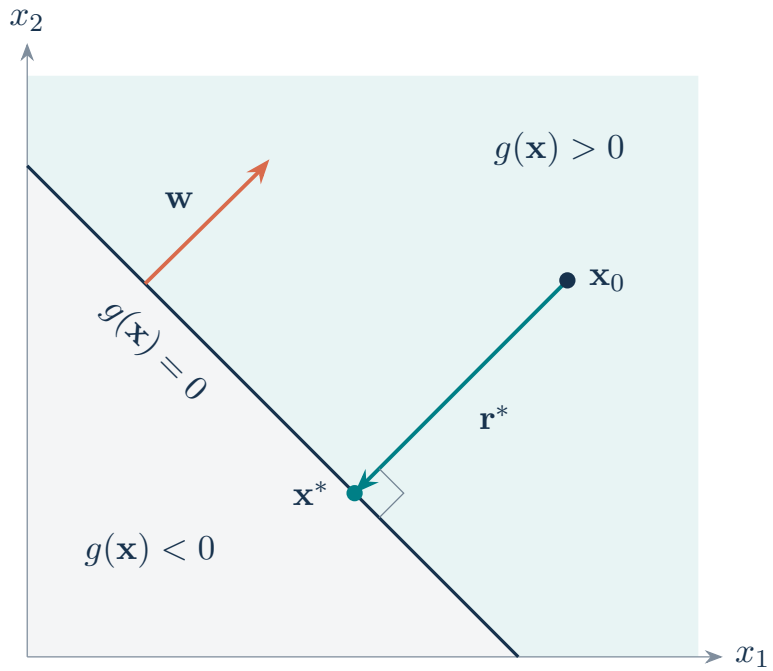
$$\boxed{\lambda^* = \frac{\mathbf{w}^\top \mathbf{x}_0 + w_0}{\|\mathbf{w}\|_2^2}.} \tag{3.9}$$

Correspondingly, the optimal $\mathbf{x}$ is

$$\mathbf{x}^* = \mathbf{x}_0 - \lambda^* \mathbf{w} = \mathbf{x}_0 - \frac{\mathbf{w}^\top \mathbf{x}_0 + w_0}{\|\mathbf{w}\|_2} \frac{\mathbf{w}}{\|\mathbf{w}\|_2}.$$

□

The search direction is $-\mathbf{w}$ (or $-\mathbf{w}/\|\mathbf{w}\|_2$ for unit norm). The negative sign is there because $\mathbf{w} = \mathbf{w}_i - \mathbf{w}_t$ points from class C_t to C_i, which is opposite to the desired search direction. The search step $(\mathbf{w}^\top \mathbf{x}_0 + w_0)/\|\mathbf{w}\|_2$ measures the distance from the input $\mathbf{x}_0$ to the target half-space. The solution is therefore the projection of $\mathbf{x}_0$ onto the separating hyperplane. If the tie rule retains C_i on the boundary, a small overshoot in the same direction is needed for a strict class change.



$$g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0, \qquad \mathbf{r}^* = -\frac{g(\mathbf{x}_0)}{\|\mathbf{w}\|_2^2}\,\mathbf{w}$$

Figure 3: The minimum Euclidean perturbation is an orthogonal projection onto the separating hyperplane. The boundary point still requires a tie rule or a small overshoot for an actual change of predicted class.

3.2 DeepFool: extending the Euclidean projection to nonlinear boundaries

For a nonlinear binary classifier, the separating boundary is $g(\mathbf{x}) = 0$. The minimum-norm construction now asks for

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{x} - \mathbf{x}_0\|_2 \qquad \text{subject to} \quad g(\mathbf{x}) = 0. \tag{3.10}$$

DeepFool [10] approaches this problem by repeatedly replacing the nonlinear boundary with its first-order approximation. Starting from $\mathbf{x}^{(0)} = \mathbf{x}_0$, expand g at the current point $\mathbf{x}^{(\ell)}$:

$$g(\mathbf{x}) \approx g(\mathbf{x}^{(\ell)}) + \nabla_{\mathbf{x}} g(\mathbf{x}^{(\ell)})^\top (\mathbf{x} - \mathbf{x}^{(\ell)}).$$

The next iterate is the closest point to the *current iterate* on this approximate boundary:

$$\begin{aligned} \mathbf{x}^{(\ell+1)} \in \arg\min_{\mathbf{x}} \; & \|\mathbf{x} - \mathbf{x}^{(\ell)}\|_2 \\ \text{subject to } & g(\mathbf{x}^{(\ell)}) + \nabla_{\mathbf{x}} g(\mathbf{x}^{(\ell)})^\top (\mathbf{x} - \mathbf{x}^{(\ell)}) = 0. \end{aligned} \tag{3.11}$$

To recognize the linear projection problem, define

$$\mathbf{w}^{(\ell)} = \nabla_{\mathbf{x}} g(\mathbf{x}^{(\ell)}), \qquad w_0^{(\ell)} = g(\mathbf{x}^{(\ell)}) - \nabla_{\mathbf{x}} g(\mathbf{x}^{(\ell)})^\top \mathbf{x}^{(\ell)}. \tag{3.12}$$

The constraint in (3.11) is then $(\mathbf{w}^{(\ell)})^\top \mathbf{x} + w_0^{(\ell)} = 0$. Apply (3.4), replacing $\mathbf{x}_0$, $\mathbf{w}$, and w_0 by $\mathbf{x}^{(\ell)}$, $\mathbf{w}^{(\ell)}$, and $w_0^{(\ell)}$, respectively:

$$\begin{aligned} \mathbf{x}^{(\ell+1)} &= \mathbf{x}^{(\ell)} - \frac{(\mathbf{w}^{(\ell)})^\top \mathbf{x}^{(\ell)} + w_0^{(\ell)}}{\|\mathbf{w}^{(\ell)}\|_2^2} \mathbf{w}^{(\ell)} \\ &= \mathbf{x}^{(\ell)} - \frac{g(\mathbf{x}^{(\ell)})}{\|\nabla_{\mathbf{x}} g(\mathbf{x}^{(\ell)})\|_2^2} \nabla_{\mathbf{x}} g(\mathbf{x}^{(\ell)}). \end{aligned}$$

Thus the binary DeepFool iteration is

$$\boxed{\mathbf{x}^{(\ell+1)} = \mathbf{x}^{(\ell)} - \frac{g(\mathbf{x}^{(\ell)})}{\|\nabla_{\mathbf{x}} g(\mathbf{x}^{(\ell)})\|_2^2} \nabla_{\mathbf{x}} g(\mathbf{x}^{(\ell)}).} \tag{3.13}$$

Each step is an exact projection onto a *local affine boundary*; the sequence does not generally give the globally nearest point on the nonlinear boundary. The formula requires a nonzero gradient. Iterations stop when the predicted class changes or an iteration limit is reached. A small overshoot can move a boundary point beyond a tie.

3.2.1 Extension to multiple classes

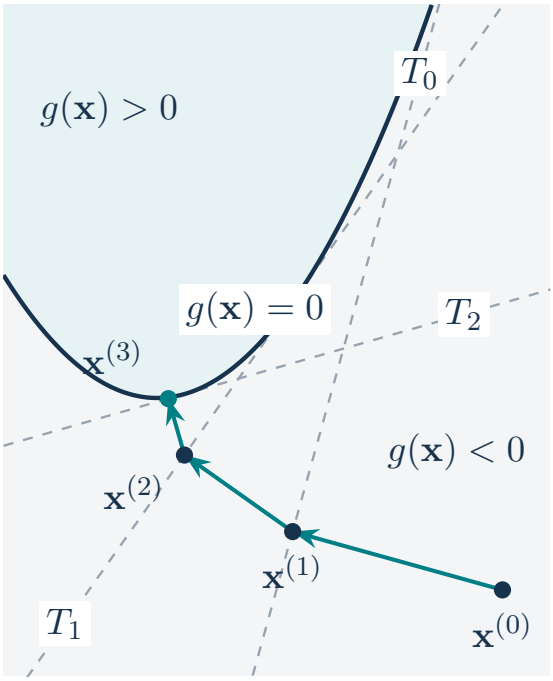
For a current prediction C_i, each competing class C_j gives a local boundary. Write $a_j^{(\ell)} = g_j(\mathbf{x}^{(\ell)}) - g_i(\mathbf{x}^{(\ell)})$ and $\mathbf{v}_j^{(\ell)} = \nabla g_j(\mathbf{x}^{(\ell)}) - \nabla g_i(\mathbf{x}^{(\ell)})$. The projection distance is $|a_j^{(\ell)}|/\|\mathbf{v}_j^{(\ell)}\|_2$. Choose the nearest boundary and use its projection:

$$\begin{aligned} j^* &\in \arg\min_{j \neq i} \frac{|a_j^{(\ell)}|}{\|\mathbf{v}_j^{(\ell)}\|_2}, \\ \mathbf{x}^{(\ell+1)} &= \mathbf{x}^{(\ell)} - \frac{a_{j^*}^{(\ell)}}{\|\mathbf{v}_{j^*}^{(\ell)}\|_2^2} \mathbf{v}_{j^*}^{(\ell)}. \end{aligned} \tag{3.14}$$

Candidates with zero normal are omitted. This nearest-competing-boundary construction is untargeted: the new class is not specified in advance.

3.3 From minimum norm to maximum allowable perturbation

The same projection question can be posed with a different norm. We first find the smallest ℓ_∞ perturbation, then use that result to derive a maximum-allowable attack with a prescribed radius.



$$g(\mathbf{x}) = x_2 - x_1^2$$

$$T_\ell : \; g(\mathbf{x}^{(\ell)}) + \nabla g(\mathbf{x}^{(\ell)})^\top (\mathbf{x} - \mathbf{x}^{(\ell)}) = 0$$

Figure 4: DeepFool repeats the linear projection construction: approximate the boundary at the current iterate, project onto the approximate boundary, and recompute the approximation. The illustrated steps approach the boundary; a crossing or overshoot is not shown.

3.3.1 Minimum ℓ_∞ perturbation

Return to $g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0$, with $g(\mathbf{x}_0) = m > 0$. Changing the measure of perturbation size gives

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{x} - \mathbf{x}_0\|_\infty \qquad \text{subject to} \quad \mathbf{w}^\top \mathbf{x} + w_0 = 0.$$

Set $\mathbf{r} = \mathbf{x} - \mathbf{x}_0$ and $b_0 = -(\mathbf{w}^\top \mathbf{x}_0 + w_0) = -m$. The equivalent problem is

$$\underset{\mathbf{r}}{\text{minimize}} \quad \|\mathbf{r}\|_\infty \qquad \text{subject to} \quad \mathbf{w}^\top \mathbf{r} = b_0.$$

Hölder's inequality supplies a lower bound on the objective. For conjugate exponents $p, q \in [1, \infty]$, with $1/p + 1/q = 1$,

$$\begin{aligned} |\mathbf{w}^\top \mathbf{r}| &\le \|\mathbf{w}\|_q \|\mathbf{r}\|_p, \\ -\|\mathbf{w}\|_q \|\mathbf{r}\|_p \le \mathbf{w}^\top \mathbf{r} &\le \|\mathbf{w}\|_q \|\mathbf{r}\|_p. \end{aligned} \tag{3.15}$$

Cauchy–Schwarz is the case $p = q = 2$. Every feasible perturbation satisfies

$$|b_0| = |\mathbf{w}^\top \mathbf{r}| \le \|\mathbf{w}\|_q \|\mathbf{r}\|_p \quad \Longrightarrow \quad \|\mathbf{r}\|_p \ge \frac{|b_0|}{\|\mathbf{w}\|_q} = \frac{m}{\|\mathbf{w}\|_q}. \tag{3.16}$$

For $p = \infty$ and $q = 1$, the lower bound is $|b_0|/\|\mathbf{w}\|_1$. We now construct a feasible vector attaining it. Take $\mathbf{r}^* = \beta \operatorname{sign}(\mathbf{w})$, where $\beta = b_0/\|\mathbf{w}\|_1$ and $\operatorname{sign}(0) = 0$. First, check the constraint:

$$\mathbf{w}^\top \mathbf{r}^* = \beta \sum_{j=1}^{d} w_j \operatorname{sign}(w_j) = \beta \sum_{j=1}^{d} |w_j| = \beta \|\mathbf{w}\|_1 = b_0.$$

Next, because $\mathbf{w} \neq \mathbf{0}$, at least one sign has magnitude one, so

$$\|\mathbf{r}^*\|_\infty = \max_j |\beta \operatorname{sign}(w_j)| = |\beta| = \frac{|b_0|}{\|\mathbf{w}\|_1}.$$

The lower bound is attained, proving optimality. Substituting $b_0 = -m$ yields

$$\boxed{\mathbf{r}^* = -\frac{m}{\|\mathbf{w}\|_1} \operatorname{sign}(\mathbf{w}), \qquad \mathbf{x}^* = \mathbf{x}_0 - \frac{\mathbf{w}^\top \mathbf{x}_0 + w_0}{\|\mathbf{w}\|_1} \operatorname{sign}(\mathbf{w}).} \tag{3.17}$$

The Euclidean solution follows the normal direction; the ℓ_∞ solution follows its coordinatewise signs. Both minimize their respective norms. An ℓ_∞ minimizer need not be unique, and a zero component of $\mathbf{w}$ can be left unchanged.

The same lower-bound argument also covers an ℓ_1 objective. Take $j^* \in \arg\max_j |w_j|$, set $r^*_{j^*} = -m/w_{j^*}$, and leave the other coordinates zero. Then $\mathbf{w}^\top \mathbf{r}^* = -m$, while $\|\mathbf{r}^*\|_1 = m/|w_{j^*}| = m/\|\mathbf{w}\|_\infty$, attaining (3.16) for $p = 1$ and $q = \infty$.

Example 3.1 (Same boundary, different perturbations)**.** Let $\mathbf{w} = (2, 1)^\top$, $w_0 = -1$, and $\mathbf{x}_0 = (1, 1)^\top$. Then $m = 2$. The Euclidean solution is $\mathbf{r}^*_2 = (-4/5, -2/5)^\top$, whose norm is $2/\sqrt{5}$. The maximum-coordinate solution is $\mathbf{r}^*_\infty = (-2/3, -2/3)^\top$, whose ℓ_∞ norm is $2/3$. An ℓ_1 solution is $\mathbf{r}^*_1 = (-1, 0)^\top$, whose ℓ_1 norm is 1. Each satisfies $\mathbf{w}^\top \mathbf{r} = -2$ and reaches the same line, but their directions reflect different definitions of perturbation size.

3.3.2 Maximum allowable perturbation with an ℓ_∞ budget

Instead of finding the smallest radius that reaches the boundary, prescribe $\eta \ge 0$ and minimize the margin within that radius:

$$\underset{\mathbf{x}}{\text{minimize}} \quad \mathbf{w}^\top \mathbf{x} + w_0 \qquad \text{subject to} \quad \|\mathbf{x} - \mathbf{x}_0\|_\infty \le \eta. \tag{3.18}$$

Using $\mathbf{x} = \mathbf{x}_0 + \mathbf{r}$ and the same b_0 gives

$$\underset{\|\mathbf{r}\|_\infty \le \eta}{\text{minimize}} \quad \mathbf{w}^\top \mathbf{r} - b_0.$$

The constant $-b_0$ does not affect the minimizer. The lower side of Hölder's inequality gives

$$\mathbf{w}^\top \mathbf{r} \ge -\|\mathbf{w}\|_1 \|\mathbf{r}\|_\infty \ge -\eta \|\mathbf{w}\|_1.$$

For $\mathbf{r}^* = -\eta \operatorname{sign}(\mathbf{w})$, the budget is satisfied and

$$\mathbf{w}^\top \mathbf{r}^* = -\eta \sum_{j=1}^{d} w_j \operatorname{sign}(w_j) = -\eta \sum_{j=1}^{d} |w_j| = -\eta \|\mathbf{w}\|_1.$$

This attains the lower bound, so

$$\boxed{\mathbf{x}^* = \mathbf{x}_0 - \eta \operatorname{sign}(\mathbf{w}), \qquad g(\mathbf{x}^*) = m - \eta \|\mathbf{w}\|_1.} \tag{3.19}$$

Choosing $\eta = m/\|\mathbf{w}\|_1$ recovers (3.17). A larger radius crosses the boundary; equality reaches it, with the predicted class determined by the tie rule. For comparison, the same calculation with Cauchy–Schwarz gives

$$\boxed{\mathbf{x}^* = \mathbf{x}_0 - \eta \frac{\mathbf{w}}{\|\mathbf{w}\|_2}, \qquad g(\mathbf{x}^*) = m - \eta \|\mathbf{w}\|_2,} \tag{3.20}$$

under an ℓ_2 budget: the bound $\mathbf{w}^\top \mathbf{r} \ge -\eta \|\mathbf{w}\|_2$ is attained by $\mathbf{r}^* = -\eta \mathbf{w}/\|\mathbf{w}\|_2$.

3.4 Fast Gradient Sign Method

The preceding sign construction extends to a differentiable loss $J(\mathbf{x}; \mathbf{w})$ that the attacker wants to maximize. Here $\mathbf{w}$ denotes the fixed model parameters, as in the lecture's loss notation. The Fast Gradient Sign Method (FGSM) [5] constructs

$$\boxed{\mathbf{x} = \mathbf{x}_0 + \eta \operatorname{sign}\big(\nabla_{\mathbf{x}} J(\mathbf{x}_0; \mathbf{w})\big).} \tag{3.21}$$

The derivative is with respect to the *input*; the model parameters are held fixed. Only one gradient evaluation is needed.

Example 3.2 (Loss for the binary linear classifier)**.** The margin $g(\mathbf{x}) = g_i(\mathbf{x}) - g_t(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0$ is positive on the original-class side. Define a loss that increases when the target's score gains on the original class:

$$J(\mathbf{x}; \mathbf{w}) = g_t(\mathbf{x}) - g_i(\mathbf{x}) = -(\mathbf{w}^\top \mathbf{x} + w_0).$$

Then $\nabla_{\mathbf{x}} J = -\mathbf{w}$, and FGSM gives

$$\mathbf{x} = \mathbf{x}_0 + \eta \operatorname{sign}(-\mathbf{w}) = \mathbf{x}_0 - \eta \operatorname{sign}(\mathbf{w}).$$

This is exactly the maximum allowable attack in (3.19). The condition $J(\mathbf{x}) > 0$ places the point strictly on the target side; $J(\mathbf{x}) = 0$ is a tie.

To derive FGSM for a nonlinear loss, write $\mathbf{x} = \mathbf{x}_0 + \mathbf{r}$ and use its first-order expansion:

$$J(\mathbf{x}_0 + \mathbf{r}; \mathbf{w}) \approx J(\mathbf{x}_0; \mathbf{w}) + \nabla_{\mathbf{x}} J(\mathbf{x}_0; \mathbf{w})^\top \mathbf{r}. \tag{3.22}$$

Let $\mathbf{h} = \nabla_{\mathbf{x}} J(\mathbf{x}_0; \mathbf{w})$. The linearized maximization and its equivalent negative minimization are

$$\begin{aligned} &\underset{\|\mathbf{r}\|_\infty \le \eta}{\text{maximize}} \quad J(\mathbf{x}_0; \mathbf{w}) + \mathbf{h}^\top \mathbf{r}, \\ &\underset{\|\mathbf{r}\|_\infty \le \eta}{\text{minimize}} \quad (-\mathbf{h})^\top \mathbf{r} - J(\mathbf{x}_0; \mathbf{w}). \end{aligned}$$

Identify the affine normal as $\widetilde{\mathbf{w}} = -\mathbf{h}$ and its constant term as $\widetilde{w}_0 = -J(\mathbf{x}_0; \mathbf{w})$. The fixed-budget result then gives

$$\mathbf{r}^* = -\eta \operatorname{sign}(\widetilde{\mathbf{w}}) = -\eta \operatorname{sign}(-\mathbf{h}) = \eta \operatorname{sign}(\mathbf{h}),$$

which proves (3.21). Equivalently, $\mathbf{h}^\top \mathbf{r} \le \eta \|\mathbf{h}\|_1$, with equality for this choice. FGSM therefore solves the linearized problem exactly; it need not maximize the original nonlinear loss.

3.4.1 The ℓ_2 version

Replace the ℓ_∞ constraint by $\|\mathbf{r}\|_2 \le \eta$:

$$\underset{\|\mathbf{r}\|_2 \le \eta}{\text{maximize}} \quad J(\mathbf{x}_0; \mathbf{w}) + \mathbf{h}^\top \mathbf{r}.$$

Cauchy–Schwarz gives $\mathbf{h}^\top \mathbf{r} \le \|\mathbf{h}\|_2 \|\mathbf{r}\|_2 \le \eta \|\mathbf{h}\|_2$. If $\mathbf{h} \neq \mathbf{0}$, choose $\mathbf{r}^* = \eta \mathbf{h} / \|\mathbf{h}\|_2$. It is feasible and satisfies $\mathbf{h}^\top \mathbf{r}^* = \eta \|\mathbf{h}\|_2^2 / \|\mathbf{h}\|_2 = \eta \|\mathbf{h}\|_2$, so it attains the upper bound. Consequently,

$$\boxed{\mathbf{x} = \mathbf{x}_0 + \eta \frac{\nabla_{\mathbf{x}} J(\mathbf{x}_0; \mathbf{w})}{\|\nabla_{\mathbf{x}} J(\mathbf{x}_0; \mathbf{w})\|_2}.} \tag{3.23}$$

If $\mathbf{h} = \mathbf{0}$, every feasible perturbation has the same linearized objective, and the zero step is a valid choice. This normalized gradient construction is often called the Fast Gradient Method (FGM); the sign operation belongs specifically to the ℓ_∞ case.

For multiclass networks, an untargeted attack commonly maximizes the original class's cross-entropy $J_i = -\log p_i$, where $p_j = \exp(g_j) / \sum_s \exp(g_s)$. A targeted attack can instead minimize $J_t = -\log p_t$, giving the step $\mathbf{x} = \mathbf{x}_0 - \eta \operatorname{sign}(\nabla_{\mathbf{x}} J_t(\mathbf{x}_0; \mathbf{w}))$.

3.5 Iterative FGSM and projected gradient attacks

An image candidate must obey pixel bounds, such as $0 \le x_j \le 1$ for normalized intensities. A single linearization may also become inaccurate far from $\mathbf{x}_0$. Iterative FGSM [8] combines image bounds with repeated linearization at the current iterate.

Let $\mathbf{h}^{(\ell)} = \nabla_{\mathbf{x}} J(\mathbf{x}^{(\ell)}; \mathbf{w})$ and let $\alpha > 0$ be the *per-step radius*. The local construction is

$$\begin{aligned} \mathbf{x}^{(\ell+1)} \in \arg\max_{\mathbf{x}} \; & J(\mathbf{x}^{(\ell)}; \mathbf{w}) + (\mathbf{h}^{(\ell)})^\top (\mathbf{x} - \mathbf{x}^{(\ell)}) \\ & \text{subject to } \|\mathbf{x} - \mathbf{x}^{(\ell)}\|_\infty \le \alpha, \qquad 0 \le \mathbf{x} \le 1. \end{aligned} \tag{3.24}$$

Terms independent of $\mathbf{x}$ can be removed. The remaining objective is $\sum_j h_j^{(\ell)} x_j$, and the feasible interval for coordinate j is

$$\max(0, x_j^{(\ell)} - \alpha) \le x_j \le \min(1, x_j^{(\ell)} + \alpha).$$

A positive $h_j^{(\ell)}$ selects the upper endpoint; a negative one selects the lower endpoint. If it is zero, retain $x_j^{(\ell)}$. These three cases are written together as

$$\mathbf{x}^{(\ell+1)} = P_{[0,1]^d}\left(\mathbf{x}^{(\ell)} + \alpha \operatorname{sign}(\mathbf{h}^{(\ell)})\right).$$

Thus clipping follows directly from maximizing the local affine loss over a product of intervals.

A per-step radius and a total attack budget serve different purposes. To enforce the prescribed radius η around the *original input*, retain the feasible set

$$\mathcal{S} = \{\mathbf{x} \in [0,1]^d : \|\mathbf{x} - \mathbf{x}_0\|_\infty \le \eta\}. \tag{3.25}$$

Its coordinate bounds are $L_j = \max(0, x_{0,j} - \eta)$ and $U_j = \min(1, x_{0,j} + \eta)$. Starting from a feasible iterate, intersecting the local step interval with $[L_j, U_j]$ gives

$$\max(L_j, x_j^{(\ell)} - \alpha) \le x_j \le \min(U_j, x_j^{(\ell)} + \alpha).$$

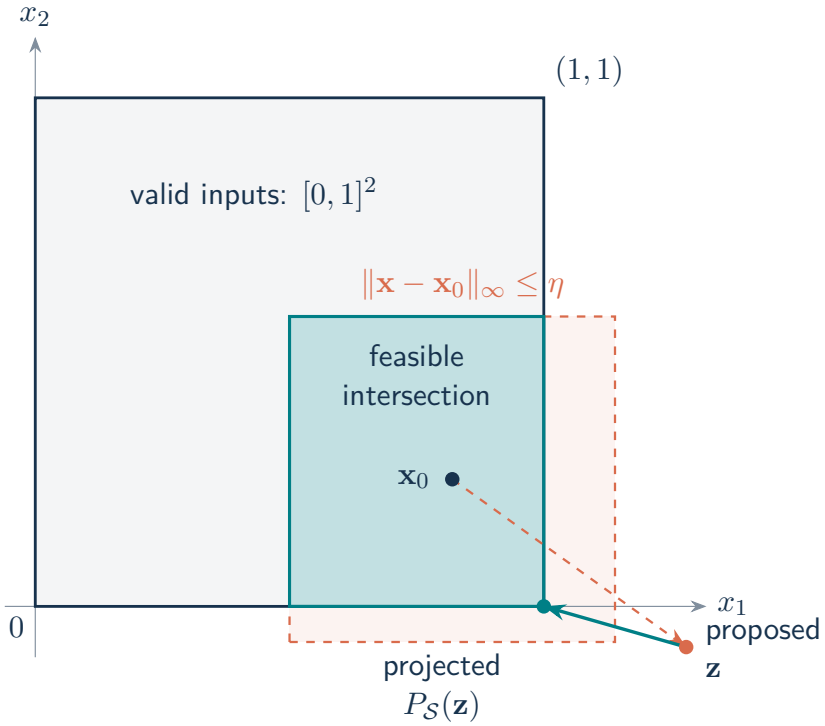
The same endpoint argument now yields the budget-preserving iteration

$$\boxed{\mathbf{x}^{(\ell+1)} = P_{\mathcal{S}}\left(\mathbf{x}^{(\ell)} + \alpha \operatorname{sign}(\nabla_{\mathbf{x}} J(\mathbf{x}^{(\ell)}; \mathbf{w}))\right).} \tag{3.26}$$

Explicitly, for a proposed point $\mathbf{z}$,

$$[P_{\mathcal{S}}(\mathbf{z})]_j = \min\{U_j, \max\{L_j, z_j\}\}. \tag{3.27}$$

Clipping only to the image range keeps pixels valid, but does not by itself enforce the distance from $\mathbf{x}_0$ after several iterations.



$$\mathcal{S} = [0,1]^2 \cap \{\mathbf{x} : \|\mathbf{x} - \mathbf{x}_0\|_\infty \le \eta\}$$

Figure 5: The feasible region is the intersection of the original perturbation budget and the valid image range. Each gradient proposal is clipped to this intersection.

The basic iterative method starts at $\mathbf{x}^{(0)} = \mathbf{x}_0$. Projected gradient attacks can instead start at a random point in $\mathcal{S}$ and use multiple restarts [9]. For a targeted attack, replace ascent on the original-class loss by descent on the target loss. In every case, check the actual predicted class and the final perturbation norm; improving the local approximation alone does not establish attack success.

3.6 Regularization-based attack

The minimum-norm and maximum-allowable formulations both impose constraints. For the binary linear classifier, these constraints lead to explicit formulas, but for a nonlinear classifier they can make the optimization difficult. The regularization formulation instead places the score margin in the objective. Following the development of Chapter 3 [3], we first examine the squared Euclidean distance and then an example where using an unsquared norm causes a difficulty.

Proposition 3.2 (Squared-distance regularization for two affine scores)**.** *For $\lambda > 0$, the unconstrained problem*

$$\min_{\mathbf{x}\in\mathbb{R}^d} F(\mathbf{x}) = \tfrac{1}{2}\|\mathbf{x}-\mathbf{x}_0\|_2^2 + \lambda(\mathbf{w}^\top\mathbf{x}+w_0) \tag{3.28}$$

has the unique solution $\mathbf{x}^* = \mathbf{x}_0 - \lambda\mathbf{w}$.

Proof. Differentiate with respect to the input, keeping λ and the classifier parameters fixed:

$$\nabla F(\mathbf{x}) = \mathbf{x} - \mathbf{x}_0 + \lambda\mathbf{w}.$$

Setting this gradient to zero gives

$$\mathbf{x}^* - \mathbf{x}_0 + \lambda\mathbf{w} = \mathbf{0} \quad\Longrightarrow\quad \boxed{\mathbf{x}^* = \mathbf{x}_0 - \lambda\mathbf{w}.}$$

The Hessian is the identity matrix, so F is strictly convex and the stationary point is its unique global minimizer. □

Substitution into the margin gives

$$g(\mathbf{x}^*) = \mathbf{w}^\top(\mathbf{x}_0 - \lambda\mathbf{w}) + w_0 = m - \lambda\|\mathbf{w}\|_2^2.$$

Consequently, the first value of λ that reaches the boundary is $\lambda^* = m/\|\mathbf{w}\|_2^2$. The corresponding distance is $\eta^* = \|\mathbf{x}^* - \mathbf{x}_0\|_2 = m/\|\mathbf{w}\|_2$. Thus λ multiplies the normal vector, whereas η measures the perturbation norm; they are generally different quantities. In a minimum-norm attack the boundary condition determines the needed distance. Here the user chooses λ, and a value that is too small need not change the class.

Example 3.3 (Why a raw margin can give an unbounded objective)**.** Replace the squared distance by an ℓ_1 norm and consider

$$\min_{\mathbf{x}\in\mathbb{R}^d} \|\mathbf{x}-\mathbf{x}_0\|_1 + \lambda(\mathbf{w}^\top\mathbf{x}+w_0).$$

Set $\mathbf{r} = \mathbf{x} - \mathbf{x}_0$. Dropping the constant λm leaves

$$\min_{\mathbf{r}\in\mathbb{R}^d} H(\mathbf{r}), \qquad H(\mathbf{r}) = \|\mathbf{r}\|_1 + \lambda\mathbf{w}^\top\mathbf{r} = \sum_{j=1}^{d}(|r_j| + \lambda w_j r_j).$$

The coordinates separate. For one coordinate, the subgradient condition is

$$0 \in \partial|r_j| + \lambda w_j, \qquad \partial|r_j| = \begin{cases}\{1\}, & r_j > 0,\\ [-1,1], & r_j = 0,\\ \{-1\}, & r_j < 0.\end{cases}$$

If $|\lambda w_j| < 1$, this condition holds only at $r_j = 0$. If $\lambda w_j = 1$, every $r_j \le 0$ minimizes that coordinate term; if $\lambda w_j = -1$, every $r_j \ge 0$ does so. Therefore:

- If $\lambda\|\mathbf{w}\|_\infty < 1$, the unique minimizer is $\mathbf{r} = \mathbf{0}$ and $H(\mathbf{r}) = 0$: there is no perturbation.

- If $\lambda\|\mathbf{w}\|_\infty = 1$, the minimum is still zero, but there are nonzero minimizers along the coordinate rays just described.
- If $\lambda\|\mathbf{w}\|_\infty > 1$, choose a coordinate with $|\lambda w_j| > 1$ and set $r_j = -s\,\mathrm{sign}(w_j)$, $s \geq 0$, leaving the other coordinates zero. Then

$$H(\mathbf{r}) = s - \lambda|w_j|s = (1 - \lambda|w_j|)s \longrightarrow -\infty.$$

  Thus the objective is unbounded below.

This example exposes two difficulties: selecting the penalty weight and continuing to reward a margin after it has already become negative. The latter can dominate an unsquared distance. A bounded input domain prevents this particular escape to infinity, but does not remove the need to choose λ or check attack success.

3.7 Carlini–Wagner attack

We now follow Chapter 3's construction of the Carlini–Wagner penalty [1, 3]: express the target constraint with an indicator, replace the indicator by a rectifier, and derive an iterative method for the resulting objective.

3.7.1 From the target constraint to a rectifier penalty

Recall the target margin and feasible region

$$\phi_t(\mathbf{x}) = \max_{j \neq t}\{g_j(\mathbf{x})\} - g_t(\mathbf{x}), \qquad \Omega_t = \{\mathbf{x} \in \mathcal{X} : \phi_t(\mathbf{x}) \leq 0\}.$$

Use the indicator function

$$\iota_{\Omega_t}(\mathbf{x}) = \begin{cases} 0, & \mathbf{x} \in \Omega_t, \\ +\infty, & \mathbf{x} \notin \Omega_t. \end{cases}$$

The minimum-norm attack is then equivalently written

$$\min_{\mathbf{x} \in \mathcal{X}} \|\mathbf{x} - \mathbf{x}_0\|_2^2 + \iota_{\Omega_t}(\mathbf{x}). \tag{3.29}$$

Squaring the distance does not change the minimizers of the constrained minimum-norm problem. A feasible point pays only its squared distance; an infeasible point receives an infinite objective value. In particular, if $\mathbf{x}_0 \in \Omega_t$, no movement is needed and the optimum is zero. Otherwise, the distance must be minimized *over the feasible points*; the indicator cannot simply be discarded.

Although (3.29) moves the target constraint into the objective, its abrupt jump to $+\infty$ remains inconvenient for numerical optimization. Introduce the rectifier

$$\zeta(s) = \max\{s, 0\} = \begin{cases} s, & s > 0, \\ 0, & s \leq 0. \end{cases} \tag{3.30}$$

Apply it to the *margin* $\phi_t(\mathbf{x})$: it is zero when the target constraint holds, and increases linearly with a positive constraint violation. Replacing the indicator by $\lambda\zeta(\phi_t(\mathbf{x}))$ gives

$$\boxed{\begin{aligned} \min_{\mathbf{x} \in \mathcal{X}} \quad & F_{\mathrm{CW}}(\mathbf{x}) = \|\mathbf{x} - \mathbf{x}_0\|_2^2 + \lambda\zeta(\phi_t(\mathbf{x})) \\ & \qquad = \|\mathbf{x} - \mathbf{x}_0\|_2^2 + \lambda \max\left\{\max_{j \neq t}\{g_j(\mathbf{x})\} - g_t(\mathbf{x}), 0\right\}. \end{aligned}} \tag{3.31}$$

Unlike the raw margin, this penalty stops decreasing once the target constraint is met. Its value is always nonnegative. Replacing an infinite penalty by a finite one is a relaxation, however: an optimizer can still prefer a smaller distance with an unmet constraint for a given λ.

Example 3.4 (The same ℓ_1 problem with a rectifier)**.** Return to the previous affine example, now minimizing

$$\|\mathbf{x}-\mathbf{x}_0\|_1+\lambda\max\{\mathbf{w}^\top\mathbf{x}+w_0,0\}.$$

Both terms are nonnegative, so the objective is bounded below by zero for every $\lambda>0$. Moreover, the distance tends to infinity as $\|\mathbf{x}\|_2\to\infty$. The continuous objective therefore attains a finite minimum on $\mathbb{R}^d$. Its minimum is zero exactly when $\mathbf{x}=\mathbf{x}_0$ and $\mathbf{w}^\top\mathbf{x}_0+w_0\le 0$; otherwise it is positive. The rectifier removes the unboundedness of the earlier raw-margin example.

3.7.2 Why the objective is convex for affine scores

If every score is affine, then

$$\phi_t(\mathbf{x})=\max_{j\neq t}\{(\mathbf{w}_j-\mathbf{w}_t)^\top\mathbf{x}+w_{j,0}-w_{t,0}\}$$

is a maximum of affine functions and hence is convex. In the multiclass case it is generally piecewise affine, rather than a single affine function. Set $h(\mathbf{x})=\max\{\phi_t(\mathbf{x}),0\}$. For $\theta\in[0,1]$,

$$\begin{aligned}h(\theta\mathbf{x}+(1-\theta)\mathbf{y})&\le\max\{\theta\phi_t(\mathbf{x})+(1-\theta)\phi_t(\mathbf{y}),0\}\\&\le\theta\max\{\phi_t(\mathbf{x}),0\}+(1-\theta)\max\{\phi_t(\mathbf{y}),0\}\\&=\theta h(\mathbf{x})+(1-\theta)h(\mathbf{y}).\end{aligned}$$

The first inequality uses convexity of ϕ_t. For the second, its right-hand side is nonnegative and is at least $\theta\phi_t(\mathbf{x})+(1-\theta)\phi_t(\mathbf{y})$, so it bounds their maximum. Thus h is convex and F_{CW} is strictly convex because of its squared-distance term. This conclusion applies on a convex input domain. For nonlinear neural-network scores, the objective generally becomes nonconvex, even though it remains bounded below by zero.

3.7.3 Computing the gradient and updating the input

Away from zero, the rectifier has derivative

$$\zeta'(s)=\begin{cases}1, & s>0,\\ 0, & s<0.\end{cases}$$

At $s=0$ it is not differentiable; its subgradient interval is $[0,1]$, and choosing zero gives the convention used in the iteration below. At the current input, let

$$i^*(\mathbf{x})\in\arg\max_{j\neq t}\{g_j(\mathbf{x})\}. \tag{3.32}$$

Suppose first that this maximizing competitor is unique and the score functions are differentiable. Then the same competitor remains active locally, so the chain rule gives, away from $\phi_t(\mathbf{x})=0$,

$$\begin{aligned}\nabla_{\mathbf{x}}\zeta(\phi_t(\mathbf{x}))&=\nabla_{\mathbf{x}}\zeta\big(g_{i^*(\mathbf{x})}(\mathbf{x})-g_t(\mathbf{x})\big)\\&=\begin{cases}\nabla g_{i^*(\mathbf{x})}(\mathbf{x})-\nabla g_t(\mathbf{x}), & \phi_t(\mathbf{x})>0,\\ \mathbf{0}, & \phi_t(\mathbf{x})<0.\end{cases}\end{aligned}$$

Adding the derivative of the squared distance therefore yields

$$\boxed{\begin{aligned}\nabla F_{\mathrm{CW}}(\mathbf{x};i^*)&=2(\mathbf{x}-\mathbf{x}_0)\\&\quad+\lambda\mathbf{1}_{\{\phi_t(\mathbf{x})>0\}}\big(\nabla g_{i^*(\mathbf{x})}(\mathbf{x})-\nabla g_t(\mathbf{x})\big).\end{aligned}} \tag{3.33}$$

The first term pulls the candidate toward the clean input; the second reduces the gap between the strongest competing score and the target score when that gap is positive. At tied competitors or hinge points, choose an appropriate subgradient (a generalized gradient in the nonlinear case). We may choose one maximizing competitor and use zero at the hinge point.

For $\mathcal{X} = \mathbb{R}^d$, initialize $\mathbf{x}^{(0)} = \mathbf{x}_0$ and repeat

$$\boxed{\begin{aligned} i_\ell^* &\in \arg\max_{j \neq t} \{g_j(\mathbf{x}^{(\ell)})\}, \\ \mathbf{x}^{(\ell+1)} &= \mathbf{x}^{(\ell)} - \alpha \nabla F_{\mathrm{CW}}(\mathbf{x}^{(\ell)}; i_\ell^*). \end{aligned}} \tag{3.34}$$

Here the gradient notation includes the chosen subgradient at nonsmooth points. Recompute i_ℓ^* at *every iteration*; keeping the competitor fixed for the entire optimization would ignore a class that later becomes stronger. For $\mathcal{X} = [0,1]^d$, project each proposed update onto that box. The two parameters have different roles:

- $\alpha > 0$ is the step size. It controls each numerical update; a fixed value need not guarantee convergence, especially at nonsmooth points.
- $\lambda > 0$ controls the balance between distance and target-constraint violation. It is adjusted to obtain successful candidates with small distortion.

Check the actual predicted class and retain successful candidates during the iterations, accounting for the tie rule when $\phi_t(\mathbf{x}) = 0$. Stopping the numerical method does not itself establish success or minimum distortion.

Confidence margin and implementation. The chapter construction above uses zero confidence margin. A positive κ replaces $\max\{\phi_t(\mathbf{x}), 0\}$ by $\max\{\phi_t(\mathbf{x}) + \kappa, 0\}$, requesting $g_t(\mathbf{x}) \geq \max_{j \neq t} g_j(\mathbf{x}) + \kappa$. The usual C&W form $\max\{\phi_t(\mathbf{x}), -\kappa\}$ differs only by the constant κ, so it has the same minimizers for a fixed λ. In the original ℓ_2 implementation, the scores are pre-softmax logits; a change of variables $\mathbf{x} = \frac{1}{2}(\tanh \mathbf{v} + \mathbf{1})$ enforces the image range, an adaptive optimizer performs the updates, and the penalty coefficient is searched. These choices refine the elementary gradient iteration while preserving the distance-plus-rectifier construction.

4 Can random noise produce an adversarial error?

The attacks considered so far use information about the classifier to choose an effective perturbation. We now follow the same geometric question for a random perturbation: can independent Gaussian noise move a correctly classified input across a decision boundary?

4.1 A random direction and a favorable half-space

Consider the affine score difference $g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0$, with $\mathbf{w} \neq \mathbf{0}$, and an input $\mathbf{x}_0$ for which $m = g(\mathbf{x}_0) > 0$. With the convention $\mathbf{w} = \mathbf{w}_i - \mathbf{w}_t$, a smaller value of g favors the competing class. Write the perturbed input as

$$\mathbf{x} = \mathbf{x}_0 + \sigma_r \mathbf{r}, \qquad \mathbf{r} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d), \quad \sigma_r > 0. \tag{4.1}$$

The components r_j are independent standard Gaussian variables. Substituting into the discriminant gives

$$g(\mathbf{x}) = m + \sigma_r \mathbf{w}^\top \mathbf{r}. \tag{4.2}$$

Consequently, $\mathbf{w}^\top \mathbf{r} < 0$ identifies a favorable direction: increasing σ_r along this direction eventually reaches the other side. If $\mathbf{w}^\top \mathbf{r} > 0$, the score instead moves away from the boundary.

Gaussian symmetry gives $\mathbb{P}[\mathbf{w}^\top \mathbf{r} < 0] = 1/2$ in every dimension. This is the probability of choosing the favorable half-space; at a fixed σ_r, crossing also requires $-\mathbf{w}^\top \mathbf{r} > m/\sigma_r$. Thus the size of the projection onto the normal, as well as its sign, determines success.

4.2 A numerical example: averaging Gaussian components

To see how cancellation appears as the dimension increases, first take $\mathbf{w} = \mathbf{1}$ and form the normalized inner product

$$Y = \frac{1}{d}\mathbf{w}^\top \mathbf{r} = \frac{1}{d}\sum_{j=1}^{d} r_j. \tag{4.3}$$

A linear combination of Gaussian variables is Gaussian. Independence gives

$$\mathbb{E}[Y] = \frac{1}{d}\sum_{j=1}^{d} \mathbb{E}[r_j] = 0, \tag{4.4}$$

$$\mathrm{Var}(Y) = \frac{1}{d^2}\sum_{j=1}^{d} \mathrm{Var}(r_j) = \frac{d}{d^2} = \frac{1}{d}. \tag{4.5}$$

Hence $Y \sim \mathcal{N}(0, 1/d)$, with standard deviation $1/\sqrt{d}$. For a fixed threshold $\epsilon > 0$, change variables $u = t\sqrt{d}$ in its Gaussian density integral:

$$\mathbb{P}[Y \geq \epsilon] = \int_{\epsilon}^{\infty} \sqrt{\frac{d}{2\pi}} \exp\left(-\frac{dt^2}{2}\right) dt \tag{4.6}$$

$$= \int_{\epsilon\sqrt{d}}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-u^2/2}\, du = 1 - \Phi(\epsilon\sqrt{d}), \tag{4.7}$$

where Φ is the standard normal cumulative distribution function. For $\epsilon = 0.1$, direct evaluation gives the following values; these are probabilities calculated from the formula, rather than simulation estimates.

Dimension d	Standard deviation $1/\sqrt{d}$	$\mathbb{P}[Y \geq 0.1]$
1	1.0000	0.460172
10	0.3162	0.375915
100	0.1000	0.158655
1000	0.0316	0.000783

The average becomes concentrated around zero. Its probability of exceeding a fixed positive threshold decreases, even though $\mathbb{P}[Y < 0] = 1/2$ remains unchanged. By symmetry, the same numerical values apply to $\mathbb{P}[Y \leq -0.1]$, the negative tail relevant to our sign convention.

4.3 The general case and an explicit Gaussian tail bound

For arbitrary weights, define

$$Y = \frac{1}{d}\sum_{j=1}^{d} w_j r_j. \tag{4.8}$$

Again the distribution is exactly Gaussian, with

$$\mu = \mathbb{E}[Y] = 0, \qquad \sigma^2 = \mathrm{Var}(Y) = \frac{1}{d^2}\sum_{j=1}^{d} w_j^2 = \frac{\|\mathbf{w}\|_2^2}{d^2}. \tag{4.9}$$

We derive the tail estimate used in Chapter 3. Let $Z \sim \mathcal{N}(0,1)$ and $a > 0$. Since $u/a \geq 1$ whenever $u \geq a$,

$$\mathbb{P}[Z \geq a] = \frac{1}{\sqrt{2\pi}} \int_a^\infty e^{-u^2/2}\, du \tag{4.10}$$

$$\leq \frac{1}{a\sqrt{2\pi}} \int_a^\infty u e^{-u^2/2}\, du \tag{4.11}$$

$$= \frac{1}{a\sqrt{2\pi}} \left[-e^{-u^2/2}\right]_a^\infty = \frac{e^{-a^2/2}}{a\sqrt{2\pi}}. \tag{4.12}$$

Now $Z = Y/\sigma$ is standard Gaussian. Substituting $a = \epsilon/\sigma = \epsilon d/\|\mathbf{w}\|_2$ gives

$$\boxed{\mathbb{P}[Y \geq \epsilon] \leq \frac{\|\mathbf{w}\|_2}{\epsilon d\sqrt{2\pi}} \exp\left(-\frac{\epsilon^2 d^2}{2\|\mathbf{w}\|_2^2}\right).} \tag{4.13}$$

The same bound holds for $\mathbb{P}[Y \leq -\epsilon]$. For a sequence of dimensions, this probability tends to zero provided $\|\mathbf{w}\|_2/d \to 0$. In particular, if $\|\mathbf{w}\|_2 \leq C\sqrt{d}$ for a constant $C > 0$, then

$$\mathbb{P}[Y \geq \epsilon] \leq \frac{C}{\epsilon\sqrt{2\pi d}} \exp\left(-\frac{\epsilon^2 d}{2C^2}\right), \tag{4.14}$$

which decreases exponentially with d for fixed $\epsilon > 0$. This includes the all-ones example, where $C = 1$. The normalization by d and the growth of the weights are essential to this concentration statement.

4.4 From a tail probability to a classification error

To apply the preceding calculation to an actual attack, substitute the classifier's threshold. The crossing event is

$$g(\mathbf{x}) < 0 \quad \Longleftrightarrow \quad Y < -\frac{m}{\sigma_r d}. \tag{4.15}$$

Since $\mathbf{w}^\top \mathbf{r} \sim \mathcal{N}(0, \|\mathbf{w}\|_2^2)$, the exact crossing probability for this affine model is

$$\boxed{\mathbb{P}[g(\mathbf{x}) < 0] = \Phi\left(-\frac{m}{\sigma_r \|\mathbf{w}\|_2}\right) = \Phi\left(-\frac{\rho_2}{\sigma_r}\right), \qquad \rho_2 = \frac{m}{\|\mathbf{w}\|_2}.} \tag{4.16}$$

Here ρ_2 is the Euclidean distance from $\mathbf{x}_0$ to the hyperplane. The threshold $m/(\sigma_r d)$ need not remain fixed when d changes; therefore the dimension effect must be assessed together with the margin and noise scale. Also, σ_r is a per-coordinate standard deviation:

$$\mathbb{E}\|\sigma_r \mathbf{r}\|_2^2 = d\sigma_r^2. \tag{4.17}$$

Thus holding σ_r fixed increases the expected total squared perturbation norm with dimension.

4.5 Geometric interpretation: the angle with the normal

Let $\mathbf{n} = \mathbf{w}/\|\mathbf{w}\|_2$ and normalize the random direction as $\mathbf{u} = \mathbf{r}/\|\mathbf{r}\|_2$. For a favorable direction, define θ as the angle between $\mathbf{u}$ and $-\mathbf{n}$. Then

$$\cos\theta = \frac{-\mathbf{w}^\top \mathbf{r}}{\|\mathbf{w}\|_2 \|\mathbf{r}\|_2} > 0. \tag{4.18}$$

If s is the Euclidean distance travelled along that direction, its component toward the hyperplane is $s\cos\theta$. Reaching the boundary requires $s\cos\theta = \rho_2$, and therefore

$$s = \frac{\rho_2}{\cos\theta} \geq \rho_2. \tag{4.19}$$

In the parameterization $\mathbf{x}_0 + \sigma_r \mathbf{r}$, the corresponding scale is

$$\sigma_r^* = \frac{s}{\|\mathbf{r}\|_2} = \frac{m}{-\mathbf{w}^\top \mathbf{r}}. \tag{4.20}$$

The travel distance s and the multiplier σ_r^* coincide only when $\mathbf{r}$ is a unit vector.

Rotational symmetry of $\mathbf{u}$ gives a precise measure of typical alignment:

$$\mathbb{E}[\mathbf{n}^\top \mathbf{u}] = 0, \qquad \mathbb{E}[(\mathbf{n}^\top \mathbf{u})^2] = \frac{1}{d}. \tag{4.21}$$

Indeed, symmetry makes the mean zero and makes all coordinate second moments equal; their sum is $\mathbb{E}\|\mathbf{u}\|_2^2 = 1$, so each is $1/d$. A random perturbation of fixed norm η consequently has a normal component with root-mean-square magnitude $\eta/\sqrt{d}$. The optimized direction $-\mathbf{n}$ uses the entire norm η to reduce the distance to the boundary. In this sense, random directions are typically nearly perpendicular to the normal in high dimension, and a typical favorable random direction needs a norm of order $\sqrt{d}$ times the optimal distance.

Random noise and adversarial perturbations

Random noise spreads its magnitude among directions, while an optimized attack concentrates it along a direction that changes the decision. This explains why a classifier may tolerate substantial random noise and still be vulnerable to a small, carefully selected perturbation.

5 Additional attack mechanisms

The last lecture slide points to three complementary ways of constructing attacks. They change different parts of the optimization problem: the admissible support of the perturbation, the treatment of the classification constraint, and the separation between direction and magnitude. In this section, g_j denotes a pre-softmax score and

$$p_j(\mathbf{x}) = \frac{\exp(g_j(\mathbf{x}))}{\sum_{a=1}^{k} \exp(g_a(\mathbf{x}))} \tag{5.1}$$

is its associated class probability. Iterations use ℓ, leaving k for the number of classes.

5.1 One Pixel Attack: sparsity as the constraint

The One Pixel Attack [15] restricts *where* the image changes. For an RGB image, let $\mathbf{r}_q \in \mathbb{R}^3$ contain the changes at pixel q, and define

$$\|\mathbf{r}\|_{0,\text{pixel}} = \#\{q : \mathbf{r}_q \neq \mathbf{0}\}. \tag{5.2}$$

A targeted search can then be written as

$$\max_{\mathbf{r}} \; p_t(\mathbf{x}_0 + \mathbf{r}) \quad \text{subject to} \quad \|\mathbf{r}\|_{0,\text{pixel}} \leq 1, \qquad \mathbf{x}_0 + \mathbf{r} \in \mathcal{X}. \tag{5.3}$$

One RGB pixel may alter three scalar coordinates; pixel sparsity therefore differs from coordinate sparsity.

The position of the modified pixel is discrete, so a gradient step on all image coordinates would not directly solve this constrained problem. The attack instead uses *differential evolution*. Each candidate stores a position and replacement colour, $\boldsymbol{\zeta} = (u, v, a_R, a_G, a_B)$. A population generates trial candidates through differences between its members, schematically

$$\widetilde{\boldsymbol{\zeta}} = \boldsymbol{\zeta}_a + F(\boldsymbol{\zeta}_b - \boldsymbol{\zeta}_c), \qquad a, b, c \text{ distinct}, \tag{5.4}$$

where $F > 0$ controls mutation. This mutation is followed by crossover with the candidate being updated. The resulting position and colour are made valid, and the corresponding image is submitted to the classifier. For the targeted objective in (5.3), the trial replaces the current candidate if it gives a larger target probability. Repeating these steps improves the population while keeping every image within the one-pixel constraint. An untargeted version can instead minimize p_i.

This is a score-based black-box method: model probabilities suffice, without parameter or gradient access. Its cost is measured in queries. Success still requires checking the predicted class. Sparsity alone does not ensure a small amplitude or visual invisibility, and unsuccessful population search does not certify robustness.

Geometrically, fixing a pixel selects a low-dimensional slice through $\mathbf{x}_0$. Searching over pixel positions selects among such slices. Thus, robustness inside a small ℓ_∞ ball and robustness to one unrestricted pixel replacement answer different questions: neither constraint contains the other in general.

5.2 ALMA: adapting the classification penalty

The Augmented Lagrangian Method for Adversarial attacks (ALMA) [13] searches for a low-distortion input satisfying the attack constraint. Let $D(\mathbf{x}, \mathbf{x}_0)$ measure distortion. For a target C_t, write

$$\min_{\mathbf{x} \in \mathcal{X}} D(\mathbf{x}, \mathbf{x}_0) \quad \text{subject to} \quad c_t(\mathbf{x}) \le 0. \tag{5.5}$$

The paper normalizes the targeted margin. For $k \ge 4$, with scores sorted in decreasing order by π, its targeted constraint is

$$c_t(\mathbf{x}) = \frac{\phi_t(\mathbf{x})}{g_{\pi_1}(\mathbf{x}) - \frac{1}{2}(g_{\pi_3}(\mathbf{x}) + g_{\pi_4}(\mathbf{x}))}, \quad \phi_t(\mathbf{x}) = \max_{j \ne t} g_j(\mathbf{x}) - g_t(\mathbf{x}). \tag{5.6}$$

With a positive denominator, normalization preserves the sign of the margin: $c_t(\mathbf{x}) > 0$ means that a competing score exceeds the target score, while $c_t(\mathbf{x}) < 0$ means that the target dominates all competitors. Thus (5.5) is a minimum-distortion attack with a normalized classification constraint. A zero denominator requires numerical handling; at a zero margin, the classifier's tie rule must be checked.

ALMA uses the penalty

$$P_2(s, \rho, \mu) = \begin{cases} \mu s + \mu \rho s^2 + \dfrac{\rho^2 s^3}{6}, & s \ge 0, \\ \dfrac{\mu s}{1 - \rho s}, & s < 0, \end{cases} \qquad \rho, \mu > 0. \tag{5.7}$$

To obtain the update, differentiate this penalty with respect to its scalar argument:

$$\partial_s P_2(s, \rho, \mu) = \begin{cases} \mu + 2\mu\rho s + \dfrac{\rho^2 s^2}{2}, & s \ge 0, \\ \dfrac{\mu}{(1 - \rho s)^2}, & s < 0. \end{cases} \tag{5.8}$$

When the constraint is violated, this derivative is at least μ; inside the feasible region, it is smaller than μ. ALMA combines this information with a moving average to update its multiplier:

$$\widehat{\mu}^{(\ell)} = \partial_s P_2\Big(c_t(\mathbf{x}^{(\ell)}), \rho^{(\ell)}, \mu^{(\ell)}\Big), \tag{5.9}$$

$$\mu^{(\ell+1)} = P_{[\mu_{\min},\mu_{\max}]}\left[\beta\mu^{(\ell)} + (1-\beta)\widehat{\mu}^{(\ell)}\right], \qquad 0 \le \beta < 1. \tag{5.10}$$

For explanation, a projected-gradient version of the subsequent step is

$$\mathcal{A}_\ell(\mathbf{x}) = D(\mathbf{x}, \mathbf{x}_0) + P_2\Big(c_t(\mathbf{x}), \rho^{(\ell)}, \mu^{(\ell+1)}\Big), \tag{5.11}$$

$$\mathbf{x}^{(\ell+1)} = P_{\mathcal{X}}\left[\mathbf{x}^{(\ell)} - \alpha_\ell \nabla_{\mathbf{x}} \mathcal{A}_\ell(\mathbf{x}^{(\ell)})\right]. \tag{5.12}$$

The implementation uses RMSProp with momentum. It increases ρ when no successful example has been found and constraint improvement is insufficient. The lowest-distortion successful iterate is retained.

This white-box approach supports several differentiable distortion measures. It is a numerical search, without a global minimum guarantee.

The chain rule explains the two contributions to this step: the gradient of the penalty is $\partial_s P_2(c_t(\mathbf{x}), \rho, \mu)\nabla c_t(\mathbf{x})$. Thus D pulls the input toward $\mathbf{x}_0$, while the weighted constraint gradient promotes the target class. The multiplier update changes their balance during the search. This differs from selecting one fixed regularization coefficient λ before optimization; μ is an estimated constraint multiplier and ρ controls the penalty's shape.

5.3 DDN: optimizing direction and radius separately

Decoupled Direction and Norm (DDN) [12] seeks a small ℓ_2 perturbation using separate direction and radius updates. Let $R_\ell > 0$ be the current radius and $0 < \gamma < 1$. For cross-entropy $J_j(\mathbf{x}; \mathbf{w}) = -\log p_j(\mathbf{x})$, with fixed model parameters $\mathbf{w}$, use

$$\mathbf{v}_\ell = \begin{cases} \nabla_{\mathbf{x}} J_i(\mathbf{x}^{(\ell)}; \mathbf{w}), & \text{untargeted}, \\ -\nabla_{\mathbf{x}} J_t(\mathbf{x}^{(\ell)}; \mathbf{w}), & \text{targeted}. \end{cases} \tag{5.13}$$

The direction update first follows the attack objective, with the gradient divided by its length. The radius is then adjusted using the current attack outcome, and the resulting perturbation is rescaled to that radius. Written with the current perturbation $\mathbf{r}^{(\ell)} = \mathbf{x}^{(\ell)} - \mathbf{x}_0$, these three operations give

$$\widetilde{\mathbf{r}}^{(\ell+1)} = \mathbf{r}^{(\ell)} + \alpha_\ell \frac{\mathbf{v}_\ell}{\|\mathbf{v}_\ell\|_2}, \tag{5.14}$$

$$R_{\ell+1} = \begin{cases} (1-\gamma)R_\ell, & \mathbf{x}^{(\ell)} \text{ satisfies the attack goal}, \\ (1+\gamma)R_\ell, & \text{otherwise}, \end{cases} \tag{5.15}$$

$$\mathbf{x}^{(\ell+1)} = P_{\mathcal{X}}\left[\mathbf{x}_0 + R_{\ell+1}\frac{\widetilde{\mathbf{r}}^{(\ell+1)}}{\|\widetilde{\mathbf{r}}^{(\ell+1)}\|_2}\right]. \tag{5.16}$$

Zero denominators require a numerical safeguard. Before box clipping, the candidate lies on a sphere centred at $\mathbf{x}_0$; clipping can reduce its norm. The original method also considers image quantization.

Failure expands the search radius; success contracts it. Gradient normalization controls direction independently of gradient magnitude. The output is the successful iterate with smallest *actual* ℓ_2 distance, since the last iterate can be unsuccessful. DDN is white-box and specialized to ℓ_2

geometry. Its adaptive radius is a search variable, rather than a prescribed budget η; evaluation at budget η additionally checks the returned distance.

For intuition, take $R_\ell = 1$ and $\gamma = 0.1$. A failure changes the radius to 1.1; a subsequent success reduces it to 0.99. These values illustrate how the radius can move across a boundary while the direction keeps changing. They do not imply convergence to the nearest boundary point for a nonlinear classifier.

6 Attacks against generative models

The preceding attacks modify a classifier's decision. A generative model instead produces a structured object, such as an image or a text sequence. Its failure cannot usually be described by a single incorrect class: an attacker may seek to alter the content, violate a conditioning instruction, reduce output quality, or recover information about the training data. The same questions remain essential: what can the attacker change, what can the attacker observe, and what counts as success?

6.1 A common formulation and the attacker-controlled variables

Write a generator as

$$\mathbf{y} = G_\theta(\mathbf{z}, \mathbf{c}; \xi), \tag{6.1}$$

where θ contains its parameters, $\mathbf{z}$ is a latent input, $\mathbf{c}$ denotes optional conditioning information, and ξ collects any additional sampling randomness. Conditioning may be an image, a text prompt, or a class label. For a deterministic generator, ξ is omitted. The symbol $\mathbf{y}$ denotes the generated output in this section, rather than a classification label.

An attack model must identify the accessible variables. Control over an uploaded image does not imply control over internal latent codes; control over a prompt does not imply access to model weights. White-box access permits gradient calculations through the accessible computation, whereas black-box access permits only the observations exposed by the interface. Changing the initial noise of an unconditional generator is also not automatically an attack: diverse outputs are expected. A security failure requires a specified property that the change violates.

The adversarial game used to train a generative adversarial network (GAN) is a learning procedure between a generator and a discriminator. The word *adversarial* in GAN training does not, by itself, describe an attack on the deployed model.

6.2 Input and latent attacks on image generators

Consider an image-to-image pipeline $F_\theta(\mathbf{x}, \mathbf{c}; \xi)$, which may include an encoder followed by a generator. With a clean image $\mathbf{x}_0$, retain the course notation $\mathbf{x} = \mathbf{x}_0 + \mathbf{r}$ and the budget $\|\mathbf{r}\|_p \le \eta$. A simple untargeted objective is

$$\max_{\substack{\|\mathbf{r}\|_p \le \eta \\ \mathbf{x}_0 + \mathbf{r} \in \mathcal{X}}} \mathbb{E}_\xi[D(F_\theta(\mathbf{x}_0 + \mathbf{r}, \mathbf{c}; \xi), F_\theta(\mathbf{x}_0, \mathbf{c}; \xi))], \tag{6.2}$$

where $\mathcal{X}$ is the valid input domain and D measures output discrepancy. Using the same random realization in the two outputs separates the perturbation's effect from ordinary sampling variation. A targeted version replaces this objective by

$$\min_{\substack{\|\mathbf{r}\|_p \le \eta \\ \mathbf{x}_0 + \mathbf{r} \in \mathcal{X}}} \mathbb{E}_\xi[D(F_\theta(\mathbf{x}_0 + \mathbf{r}, \mathbf{c}; \xi), \mathbf{y}_t)], \tag{6.3}$$

where $\mathbf{y}_t$ is the desired output. The distinction is the same as for classifiers: increase a discrepancy for an untargeted attack, or decrease a target discrepancy for a targeted attack. If this objective is differentiable, the construction follows the earlier projected-gradient method: compute its input gradient, take an ascent or descent step as appropriate, and project back onto the input domain and perturbation constraint. An alternative penalty formulation combines the target discrepancy with $\lambda\|\mathbf{r}\|_p$, where λ controls the cost of changing the input.

The discrepancy should express the intended failure. Pixelwise squared error measures numerical changes; a distance between task-relevant features can measure a different notion of content change. Neither is automatically equivalent to a perceptual or semantic failure. A small input norm likewise does not guarantee imperceptibility.

For an encoder–decoder model, one may instead try to move the encoded representation towards that of a target image. Such attacks were investigated for variational autoencoders (VAEs) and VAE–GAN architectures by Kos et al. [7]. In a stochastic VAE, the encoder defines a distribution over latent codes, so the objective must specify whether it compares distribution parameters or averages over sampled codes. Directly optimizing $\mathbf{z}$ is a different threat model, valid only if the attacker can supply or modify that latent input.

6.3 Diffusion models: conditioning and sampling randomness

A diffusion generator constructs its output through a sequence of denoising steps. An image-editing application may first encode the supplied image, add noise at a chosen level, and then denoise under a text condition. A small perturbation of the supplied image can therefore influence later generation even when the image itself still looks similar. Salman et al. [14] study this mechanism as a way to disrupt unwanted image editing: their methods target either the image encoder or the diffusion-based editing process.

Equation (6.2) provides a useful general abstraction, rather than the exact objective of every diffusion attack. For a stochastic editor, optimizing a single seed can produce an attack that works only for that seed. The expectation can be approximated by averaging over several random realizations, and evaluation should use fresh realizations. Robustness to different prompts, image transformations, or samplers is an additional claim requiring separate evidence. Gradients through many denoising steps can also be expensive; a surrogate or truncated computation changes the attack approximation and should be reported.

Text-only generation has a different input space. If the attacker controls a prompt but cannot supply an image or a continuous embedding, the admissible changes are token edits, not arbitrary pixel perturbations. The perturbation budget must then be defined accordingly, for example by the number of permitted token changes.

6.4 Language models: jailbreaks and prompt injection

An autoregressive language model assigns a probability to a response $\mathbf{y} = (y_1, \ldots, y_T)$ given a context $\mathbf{c}$:

$$p_\theta(\mathbf{y} \mid \mathbf{c}) = \prod_{t=1}^{T} p_\theta(y_t \mid \mathbf{c}, y_1, \ldots, y_{t-1}). \tag{6.4}$$

Adversarial changes to the context can redirect generation without changing θ. Two related but distinct settings are useful to separate.

A *jailbreak* seeks to defeat the model's intended safety restrictions. The attacker often controls a direct user prompt. For a prescribed response, one possible search objective is its negative log-likelihood: taking the logarithm of the product above turns this into a sum of token losses. Candidate prompt edits can then be compared by how much they reduce that loss; the generated

response must still be checked against the attack goal. Zou et al. [17] investigate automatically optimized adversarial suffixes and their transfer between models. The continuous-gradient intuition from image attacks remains useful, but an actual text interface accepts discrete tokens; an arbitrary embedding perturbation need not correspond to a realizable prompt.

A *prompt injection* attempts to make an application treat attacker-controlled content as authoritative instructions. In an indirect injection, the attacker may control a retrieved document or web page rather than the user's request. Greshake et al. [6] examine this setting in applications built around language models. The defining issue is a trust boundary between instructions and external data. A successful injection might merely redirect a summary or corrupt a workflow, without requesting prohibited content. Conversely, a direct jailbreak need not involve retrieved material. These categories can overlap, but they describe different goals and control surfaces.

6.5 Training-time poisoning and backdoors

Inference attacks keep model parameters fixed. A poisoning attacker instead influences training data or the training process. A backdoor is a more specific goal: preserve ordinary behavior while making a trigger cause a chosen deviation. The trigger may be associated with an input pattern or condition, depending on the architecture. Chou et al. [4] demonstrate this distinction for diffusion models: normal inputs can retain useful generation while a trigger redirects the output.

The attacker therefore needs training-stage influence, or the ability to distribute a compromised model or adaptation. Merely finding a problematic prompt in an otherwise fixed model is not evidence of poisoning. This gives two evaluation conditions: without the trigger, generation should retain its ordinary quality; with the trigger, the attacker seeks the prescribed deviation. Both must therefore be measured. High ordinary quality alone cannot rule out a backdoor.

6.6 Privacy attacks and evaluation criteria

A membership-inference attacker asks whether a candidate example belonged to the training set. A training-data extraction attacker attempts to recover training content itself. These objectives concern confidentiality, rather than input robustness, and do not require malicious changes to training. The threat model must state which queries and outputs the attacker can access and what prior knowledge is available. Carlini et al. [2] show that diffusion models can regenerate memorized training images under suitable querying conditions.

Similarity to a training example is not, by itself, proof of membership or memorization: common structures can also appear in unseen examples. Membership evaluation needs a non-member comparison and a controlled false-positive rate; extraction evaluation needs a clearly defined match criterion. For generative robustness more broadly, report the attacker's control, perturbation or query budget, task-specific success criterion, ordinary output quality, and variation across random seeds. Classification accuracy alone cannot summarize these different failures.

7 Comparing attacks and interpreting the results

The methods in this course answer different questions. A fixed-budget attack tests what can happen within a prescribed region. A minimum-distortion attack estimates how close a failure may be. A query-based attack adds an information and resource constraint. These distinctions should remain visible in reported results.

Method	Main constraint or objective	Characteristic mechanism
FGSM	Fixed ℓ_∞ budget	One input-gradient sign step.
I-FGSM / PGD	Fixed norm budget and input domain	Repeated gradient steps with projection; restarts can broaden the search.
DeepFool	Small ℓ_2 perturbation	Repeated projection onto locally approximated boundaries.
C&W	Distortion plus success penalty	Optimize a logit-margin penalty and search its weight.
One-Pixel	Sparse pixel support	Score-query search over locations and replacement colors.
ALMA	Small distortion with attack constraint	Adapt multipliers and penalty strength.
DDN	Small ℓ_2 perturbation	Update direction and perturbation radius separately.

Table 2: A comparison of the roles of the attacks discussed in the notes. Access assumptions and norm variants must be specified for an experiment.

7.1 A consistent classification protocol

State the model, preprocessing, data split, norm, radius, input range, target selection rule, and access assumptions. Record the number of iterations, step-size schedule, random restarts, and queries. A radius measured before input normalization must be converted correctly if optimization occurs after normalization. If the deployed system includes preprocessing or randomness, the evaluated attack should account for that complete system.

For examples indexed by $n = 1, \dots, N$, let i_n be the true class, $\mathbf{x}_{0,n}$ the clean input, and $\mathbf{x}_n^{\text{adv}}$ a valid attack candidate. Let $\mathcal{I} = \{n : \widehat{\imath}(\mathbf{x}_{0,n}) = i_n\}$ be the clean-correct subset. An untargeted attack success rate conditioned on initial correctness is

$$\text{ASR} = \frac{1}{|\mathcal{I}|} \sum_{n \in \mathcal{I}} \mathbf{1}\{\widehat{\imath}(\mathbf{x}_n^{\text{adv}}) \neq i_n\}. \tag{7.1}$$

For targeted success, replace the event by $\widehat{\imath}(\mathbf{x}_n^{\text{adv}}) = t_n$, with $t_n \neq i_n$. The denominator must be reported; an already incorrect clean prediction should not count as a newly induced error. For minimum-distortion methods, report both success rate and the distribution of successful perturbation norms, not just an average that silently excludes failures.

Within a fixed threat set, the worst-case robust accuracy asks whether each example is correct for *every* allowed perturbation. Searching with a finite collection of attacks explores only part of that set. The resulting empirical robust accuracy is consequently an upper bound on the true worst-case robust accuracy, assuming the evaluation includes the clean input and all candidates are valid. Failing to find an adversarial example is not a robustness certificate. Success, in contrast, is a concrete counterexample for that input.

7.2 Questions for self-study

1. **Distance to a linear boundary.** A clean binary input has margin $m = 3$ and normal $\mathbf{w} = (3, 4)^\top$. Compute the minimum ℓ_2 and ℓ_∞ distances to its boundary. Which norm permits crossing with radius $\eta = 0.5$? Work in $\mathbb{R}^2$ without input-box constraints and distinguish reaching the boundary from crossing it.
2. **An untargeted multiclass constraint.** For three classes, suppose the perturbed scores are $(g_i, g_a, g_b) = (2, 3, -1)$, where C_i is the original class. Has an untargeted attack succeeded? Compare the correct maximum-based constraint with the incorrect minimum-based version.
3. **Clipping and the total budget.** In one dimension, take $x_0 = 0.95$, $\eta = 0.10$, and two positive steps of size $\alpha = 0.08$. Compare projection onto $[0, 1]$ with projection onto $[0, 1] \cap [x_0 - \eta, x_0 + \eta]$. Then restart at x_0 and repeat with two negative steps. Give both intermediate iterates and their final perturbations.
4. **Stochastic generative models.** Explain why a large output change in a stochastic generator is not, by itself, evidence of an attack. What should be held fixed or averaged over? Describe a comparison that isolates the effect of an allowed input perturbation and a way to evaluate its success across random draws.
5. **FGSM and two projected steps in two dimensions.** Let $\mathbf{x}_0 = (0.90, 0.20)^\top \in [0, 1]^2$ and use an ℓ_∞ budget $\eta = 0.15$. Consider the affine attack loss $J(\mathbf{x}; \mathbf{w}) = 2x_1 - x_2$, which is to be maximized. The parameters $\mathbf{w}$ are fixed. Compute the feasible set, the raw FGSM candidate, and its projection. Then start again at $\mathbf{x}_0$ and perform two projected sign-gradient steps with $\alpha = 0.10$. At each step, give the raw candidate and projected iterate. What happens if only $[0, 1]^2$ clipping is used? Compute the feasible loss increase and explain whether the projected FGSM point is an exact maximizer for this example.
6. **DeepFool on a curved boundary.** Consider the binary score difference $g(\mathbf{x}) = x_1^2 + x_2^2 - 1$ and $\mathbf{x}^{(0)} = \mathbf{x}_0 = (2, 0)^\top$ in $\mathbb{R}^2$, without input-box constraints. The original class occupies $g > 0$. Perform two binary DeepFool updates using the squared gradient norm in the denominator. Compute the remaining score after each update. Compare the accumulated perturbation with the exact distance to the boundary. Finally, multiply that accumulated perturbation by $c > 1$: find the value of c that first reaches the boundary, and decide whether $c = 1.04$ produces a strict class change.
7. **Gaussian noise versus an aligned perturbation.** A binary affine classifier in $\mathbb{R}^4$ has clean margin $m = 2$ and $\mathbf{w} = (2, 2, 2, 2)^\top$. Let $\mathbf{x} = \mathbf{x}_0 + \sigma_r \mathbf{r}$ with $\mathbf{r} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_4)$ and $\sigma_r = 0.30$. Find the distribution of the perturbed margin and its exact probability of becoming negative, expressed using Φ. Compute $\mathbb{E}\|\sigma_r \mathbf{r}\|_2^2$ and set $\eta = \sqrt{\mathbb{E}\|\sigma_r \mathbf{r}\|_2^2}$. Construct the deterministic perturbation of norm η that decreases the margin most, and check whether it crosses the boundary. Does matching this root-mean-square norm make Gaussian noise a fixed-budget attack? Explain the distinction.

References

[1] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *IEEE Symposium on Security and Privacy*, pages 39–57, 2017. doi: 10.1109/SP.2017.49. URL https://arxiv.org/abs/1608.04644.

[2] Nicholas Carlini, Jamie Hayes, Milad Nasr, Matthew Jagielski, Vikash Sehwag, Florian Tramèr, Borja Balle, Daphne Ippolito, and Eric Wallace. Extracting training data from diffusion models. In *32nd USENIX Security Symposium*, pages 5253–5270. USENIX Association, 2023. URL https://www.usenix.org/conference/usenixsecurity23/presentation/carlini.

[3] Stanley H. Chan. Chapter 3: Adversarial Attack. Purdue University course notes, 2018. URL https://engineering.purdue.edu/ChanGroup/ECE595/files/chapter3.pdf. Supplied 29-page chapter; copyright year printed in the document.

[4] Sheng-Yen Chou, Pin-Yu Chen, and Tsung-Yi Ho. How to backdoor diffusion models? In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4015–4024, 2023. URL https://openaccess.thecvf.com/content/CVPR2023/html/Chou_How_to_Backdoor_Diffusion_Models_CVPR_2023_paper.html.

[5] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1412.6572.

[6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023. doi: 10.48550/arXiv.2302.12173. URL https://arxiv.org/abs/2302.12173.

[7] Jernej Kos, Ian Fischer, and Dawn Song. Adversarial examples for generative models. *arXiv preprint arXiv:1702.06832*, 2017. doi: 10.48550/arXiv.1702.06832. URL https://arxiv.org/abs/1702.06832.

[8] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. Adversarial examples in the physical world. *arXiv preprint arXiv:1607.02533*, 2017. URL https://arxiv.org/abs/1607.02533. Revised version; first submitted in 2016.

[9] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations*, 2018. URL https://arxiv.org/abs/1706.06083.

[10] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, and Pascal Frossard. DeepFool: A simple and accurate method to fool deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2574–2582, 2016. URL https://www.cv-foundation.org/openaccess/content_cvpr_2016/html/Moosavi-Dezfooli_DeepFool_A_Simple_CVPR_2016_paper.html.

[11] Ana Nicolae and Vlad Vasilescu. Artificial Intelligence III: Advanced Deep Learning Methods. Lecture 1: Attack Mechanisms. BIOSINF Master Program, National University of Science and Technology POLITEHNICA Bucharest, 2026. October. Course slides supplied with this project.

[12] Jérôme Rony, Luiz G. Hafemann, Luiz S. Oliveira, Ismail Ben Ayed, Robert Sabourin, and Eric Granger. Decoupling direction and norm for efficient gradient-based L_2 adversarial attacks and defenses. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4322–4330, 2019. URL https://openaccess.thecvf.com/content_CVPR_2019/html/Rony_Decoupling_Direction_and_Norm_for_Efficient_Gradient-Based_L2_Adversarial_Attacks_CVPR_2019_paper.html.

[13] Jérôme Rony, Eric Granger, Marco Pedersoli, and Ismail Ben Ayed. Augmented lagrangian adversarial attacks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7738–7747, 2021. URL https://openaccess.thecvf.com/content/ICCV2021/html/Rony_Augmented_Lagrangian_Adversarial_Attacks_ICCV_2021_paper.html.

[14] Hadi Salman, Alaa Khaddaj, Guillaume Leclerc, Andrew Ilyas, and Aleksander Madry. Raising the cost of malicious AI-powered image editing. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 29894–29918. PMLR, 2023. URL https://proceedings.mlr.press/v202/salman23a.html.

[15] Jiawei Su, Danilo Vasconcellos Vargas, and Kouichi Sakurai. One pixel attack for fooling deep neural networks. *IEEE Transactions on Evolutionary Computation*, 23(5):828–841, 2019. doi: 10.1109/TEVC.2019.2890858. URL https://arxiv.org/abs/1710.08864.

[16] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *International Conference on Learning Representations*, 2014. URL https://arxiv.org/abs/1312.6199.

[17] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023. doi: 10.48550/arXiv.2307.15043. URL https://arxiv.org/abs/2307.15043.